QUANTUM SECURITY INTELLIGENCE

Security & Post-Quantum Readiness Audit Report

bitgo-mpc-en @ 3c892fd6a388 · BitGo — public repository (passive review)

Organization	QSI
Report date	2026-11-04 18:37 UTC
Report version	1
Audit run	c5ba2a31-05e5-4772-8847-ee34fb1b03a9 · Document audit · 2026-11-04 18:37 UTC
Analyzer engine	qsi-engine 1.3.0
Risk profile	Crypto exchange, custodian or wallet
Content SHA-256	`38bce0925d7c971c44f19b16d5f05ed81e6d3158320e060b824044b1cb6ac506`
Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json)	`5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`

Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

Executive summary

10	0	0	9	0
Findings in total	Critical	High	Medium	Low

Classification	Count
Confirmed technical vulnerability	0
Known vulnerable component	0
Confirmed configuration error	0
Potential cryptographic risk	0
Long-term post-quantum risk	9
Improvement recommendation	0
Insufficient data	1

Top 5

ID	Severity	Detailed findings	Priority score
QSI-2026-11792	Medium	Quantum-vulnerable RSA: key wrapping and data encryption at rest (1 file(s)) confirmed by the analyst	43.4
QSI-2026-11786	Medium	Quantum-vulnerable Ed25519: transaction and message signing (2 file(s)) confirmed by the analyst	41.8
QSI-2026-11787	Medium	Quantum-vulnerable ECDSA: transaction and message signing (10 file(s)) confirmed by the analyst	41.8
QSI-2026-11788	Medium	Quantum-vulnerable ECDSA: threshold signing and MPC protocols (10 file(s)) confirmed by the analyst	41.8
QSI-2026-11789	Medium	Quantum-vulnerable Paillier: threshold signing and MPC protocols (14 file(s)) confirmed by the analyst	41.8

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Analyst review: 9 confirmed, 0 not applicable, 0 dismissed as false positives.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography. Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Scope and boundaries

Mode: Document audit

Repository: https://github.com/BitGo/BitGoJS @ 3c892fd6a388da22a046a8470f0e1f35999e4ed4 — evidence links open the lines at this commit.

Supplied inputs

271 files, 2022968 bytes in total, 0 redaction(s); the per-file SHA-256 list is in appendix C.

Methodology

The audit was performed by the QSI engine 1.3.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

Coverage of checks

Area	Status	Count	Examples
dependencies	checked	3	modules/sdk-core/package.json: 36 dependencies parsed (13 pinned); modules/sdk-lib-mpc/package.json: 30 dependencies parsed (22 pinned); package.json: 59 dependencies parsed (7 pinned)
source_code	checked	268	modules/sdk-core/src/account-lib/baseCoin/baseKeyPair.ts: 30 lines scanned with 16 rule patterns and 145 registry patterns; modules/sdk-core/src/account-lib/baseCoin/baseTransaction.ts: 101 lines scanned with 16 rule patterns and 145 registry patterns; modules/sdk-core/src/account-lib/baseCoin/baseTransactionBuilder.ts: 222 lines scanned with 16 rule patterns and 145 registry patterns
keys	error (not checked)	16	modules/sdk-core/src/bitgo/trading/network/encrypt.ts line 19: public key could not be parsed (ValueError); modules/sdk-core/src/bitgo/trading/network/encrypt.ts line 45: public key could not be parsed (ValueError); modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 8: OpenPGP key block could not be parsed
certificates_and_keys	checked	2	modules/sdk-core/src/bitgo/trading/network/encrypt.ts: certificates/keys parsed; modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts: certificates/keys parsed
entropy	checked	1	random-number generation: 268 code files read with 0 macro definitions from the tree; 0 ineffective guard(s), 0 weak random-bytes provider(s), 0 weak key-generation site(s)
osv_advisories	checked	1	32 pinned packages queried at OSV.dev; 0 affected

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

Assets examined and not examined

Supplied inputs: 273 (input files: C)

Not checked

- keys: modules/sdk-core/src/bitgo/trading/network/encrypt.ts line 19: public key could not be parsed (ValueError)
- keys: modules/sdk-core/src/bitgo/trading/network/encrypt.ts line 45: public key could not be parsed (ValueError)
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 8: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 12: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 17: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 18: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 21: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 22: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 27: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 28: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 31: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 32: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 43: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 44: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 47: OpenPGP key block could not be parsed
- keys: modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 48: OpenPGP key block could not be parsed

Architecture overview

Static review of the source tree bitgo-mpc-en at 3c892fd6a388da22a046a8470f0e1f35999e4ed4. 271 files analysed; of 272 files, 0 are test, fuzzing, benchmark or example code and were left out, 1 are not read by any analyzer, 0 exceed the per-file limit and 0 did not fit the total limit. The review was restricted to 5 path(s) of the repository: modules/sdk-core/package.json, modules/sdk-core/src, modules/sdk-lib-mpc/package.json, modules/sdk-lib-mpc/src, package.json; 5940 files outside them were not read. Files no analyzer reads include: modules/sdk-lib-mpc/src/openssl/README.md. No requests were sent to the owner's systems.

Overall security posture

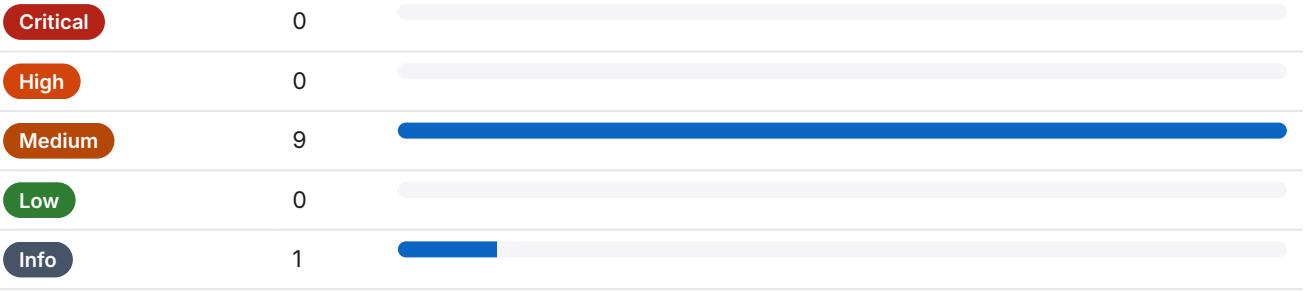
100
Indicative posture index (0–100)

73.1%
Quantum-vulnerable share of inventory

93
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

Findings by severity



ID	Detailed findings	Classification	Severity	Confidence
QSI-2026-11792	Quantum-vulnerable RSA: key wrapping and data encryption at rest (1 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11786	Quantum-vulnerable Ed25519: transaction and message signing (2 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11787	Quantum-vulnerable ECDSA: transaction and message signing (10 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11788	Quantum-vulnerable ECDSA: threshold signing and MPC protocols (10 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11789	Quantum-vulnerable Paillier: threshold signing and MPC protocols (14 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11790	Quantum-vulnerable Threshold ECDSA: threshold signing and MPC protocols (16 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11794	Quantum-vulnerable X25519: threshold signing and MPC protocols (2 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11791	Quantum-vulnerable ECDSA: role not determined (2 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11793	Quantum-vulnerable RedDSA: threshold signing and MPC protocols (2 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary
QSI-2026-11795	No post-quantum capable cryptographic library detected	Insufficient data	Info	Medium

Detailed findings

QSI-2026-11792 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable RSA: key wrapping and data encryption at rest (1 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Correct and specific enough to act on: modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts imports an SPKI PEM public key through WebCrypto with { name: 'RSA-OAEP', hash: 'SHA-256' } and usage ['encrypt'], deliberately reading window.crypto in the browser because crypto-browserify does not implement it. So this is RSA-OAEP key wrapping toward a server key in the trading network, and the ciphertext is transmitted and may be retained — harvest-now-decrypt-later, and the first candidate for a hybrid KEM because it is a client-to-server channel the vendor controls at both ends.

2 reference(s) to RSA across 1 file(s), e.g. modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts:33. Role, judged from paths, function names and nearby code: key wrapping and data encryption at rest. Keys or data encrypted to these public keys are stored: anyone who copies them now decrypts them once a CRQC exists (harvest now, decrypt later).

Affected asset	source-code:RSA:key_wrapping
Data source	modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	43.4 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts:33 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts · line 33 · SHA-256 af9a7b3e6c70a3ae… · File 77177724e5cbd064…

```
32 |     {
33 |       name: 'RSA-OAEP',
34 |       hash: 'SHA-256',
```

file_excerpt · modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts · line 57 · SHA-256 bdce29586696e5a9… · File 77177724e5cbd064…

```
56 |     {
57 |       name: 'RSA-OAEP',
58 |     },
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Wrap keys with ML-KEM or a hybrid KEM (X25519 + ML-KEM-768) and re-wrap the stored keys; data that may already have been copied needs its own assessment.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11786 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Ed25519: transaction and message signing (2 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** account-lib/baseCoin/ed25519KeyPair.ts and baseCoin/index.ts — transaction signing on the Ed25519 chains. Small row count, same migration class as the secp256k1 signing rows.

2 reference(s) to Ed25519 across 2 file(s), e.g. modules/sdk-core/src/account-lib/baseCoin/ed25519KeyPair.ts:16. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:Ed25519:transaction_signing
Data source	modules/sdk-core/src/account-lib/baseCoin/ed25519KeyPair.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-core/src/account-lib/baseCoin/ed25519KeyPair.ts:16 and inspect the quoted lines;
the evidence lists further files.
```

EVIDENCE

file_excerpt · modules/sdk-core/src/account-lib/baseCoin/ed25519KeyPair.ts · line 16 · SHA-256 e8482eb972e2150f… · File 7ea2c42fb769971f…

```
   15 |
   16 | export abstract class Ed25519KeyPair implements BaseKeyPair {
   17 |   protected keyPair: DefaultKeys;
```

file_excerpt · modules/sdk-core/src/account-lib/baseCoin/index.ts · line 6 · SHA-256 af80a0b2e1e69e65… · File ba3540f3b0cc2c17…

```
    5 | export { BaseKeyPair } from './baseKeyPair';
    6 | export { Ed25519KeyPair } from './ed25519KeyPair';
    7 | export { TransactionType, StakingOperationTypes, AddressFormat, DotAddressFormat,
DotAssetTypes } from './enum';
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11787 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: transaction and message signing (10 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Two roles, both correct. Transaction and message signing: account-lib/baseCoin/secp256k1ExtendedKeyPair.ts, account-lib/util/crypto.ts, bitgo/ecdh.ts and bitgo/utils/tss/addressVerification.ts. Threshold signing and MPC: account-lib/mpc/curves/index.ts, account-lib/mpc/tss/ecdsa/ecdsa.ts and bitgo/utils/tss/ecdsa/{base,ecdsa}.ts. A CRQC derives the private key from a public key that is already on chain after the first spend, so for a custodian the exposure is proportional to how much of its address set has ever been spent from.

14 reference(s) to ECDSA across 10 file(s), e.g. modules/sdk-core/src/account-lib/baseCoin/secp256k1ExtendedKeyPair.ts:1. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already

known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:ECDSA:transaction_signing
Data source	modules/sdk-core/src/account-lib/baseCoin/secp256k1ExtendedKeyPair.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-core/src/account-lib/baseCoin/secp256k1ExtendedKeyPair.ts:1 and inspect the quoted
lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · modules/sdk-core/src/account-lib/baseCoin/secp256k1ExtendedKeyPair.ts · line 1 · SHA-256 6ddb812639dd1256… · File 411521d568ce8dee…

```
    1 | import { bip32, BIP32Interface, ECPair, ECPairInterface } from '@bitgo/secp256k1';
    2 | import * as Crypto from '../util/crypto';
```

file_excerpt · modules/sdk-core/src/account-lib/util/crypto.ts · line 2 · SHA-256 1b12201a01dddcd4… · File d619311bd82ddd74…

```
    1 | import { networks } from '@bitgo/utxo-lib';
    2 | import { bip32, ECPair } from '@bitgo/secp256k1';
    3 | import * as hex from '@stablelib/hex';
```

file_excerpt · modules/sdk-core/src/bitgo/ecdh.ts · line 16 · SHA-256 a1915605806bdb59… · File 817c83e398ed1f0c…

```
   15 | import * as assert from 'assert';
   16 | import * as secp256k1 from 'secp256k1';
   17 | import { ECPairInterface, BIP32Interface, bip32, ECPair } from '@bitgo/utxo-lib';
```

file_excerpt · modules/sdk-core/src/bitgo/ecdh.ts · line 60 · SHA-256 7e080a30d6ce1730… · File 817c83e398ed1f0c…

```
   59 |   //                see discussion here https://github.com/bitcoin-core/secp256k1/issues/
352
   60 |   const buffer = Buffer.from(secp256k1.publicKeyTweakMul(publicKey, privateKey))
   61 |     // remove leading parity bit
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/addressVerification.ts · line 60 · SHA-256 796a023c4b3818bd… · File 84e20697851f775c…

```
   59 | > & {
   60 |   keyCurve: 'secp256k1' | 'ed25519';
   61 | };
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/addressVerification.ts · line 90 · SHA-256 eaa6e881ef2e6ee8… · File 84e20697851f775c…

```
   89 |
   90 |   const MPC = params.keyCurve === 'secp256k1' ? new Ecdsa() : await
EDDSAMethods.getInitializedMpcInstance();
   91 |   const derivedPublicKey = MPC.deriveUnhardened(commonKeychain, derivationPath);
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/addressVerification.ts · line 94 · SHA-256 5fe68310e693027a… · File 84e20697851f775c…

```
   93 |   // secp256k1 expects 33 bytes; ed25519 expects 32 bytes
   94 |   const publicKeySize = params.keyCurve === 'secp256k1' ? 33 : 32;
   95 |   const publicKeyOnly = Buffer.from(derivedPublicKey, 'hex').subarray(0,
publicKeySize).toString('hex');
```

file_excerpt · modules/sdk-core/src/bitgo/utils/util.ts · line 196 · SHA-256 088bf5b0ec4051f2… · File ad33cca15d17c290…

```
  195 |
  196 |     const pubKey = ethUtil.ecrecover(Buffer.from(msgHash, 'hex'), v, r, s);
  197 |     return ethUtil.bufferToHex(ethUtil.pubToAddress(pubKey));
```

file_excerpt · modules/sdk-core/src/bitgo/wallet/iWallet.ts · line 1403 · SHA-256 fea5f8cfea544293… · File d66cfa8b23933fbf…

```
 1402 |   buildSignMessageRequest(params: WalletSignMessageOptions): Promise<TxRequest>;
 1403 |   signTypedData(params: WalletSignTypedDataOptions): Promise<SignedMessage>;
 1404 |   signAndSendMessageTxRequest(params: SignAndSendMessageTxRequestOptions):
Promise<SignedMessage>;
```

file_excerpt · modules/sdk-core/src/bitgo/wallet/wallet.ts · line 2517 · SHA-256 286358dd24445d05… · File 2cee1dfbd5165bde…

```
 2516 |    */
 2517 |   async signTypedData(params: WalletSignTypedDataOptions): Promise<SignedMessage> {
 2518 |     if (!this.baseCoin.supportsSigningTypedData()) {
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11788 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: threshold signing and MPC protocols (10 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Two roles, both correct. Transaction and message signing: account-lib/baseCoin/secp256k1ExtendedKeyPair.ts,

> account-lib/util/crypto.ts, bitgo/ecdh.ts and bitgo/utils/tss/addressVerification.ts. Threshold signing and MPC: account-lib/mpc/curves/index.ts, account-lib/mpc/tss/ecdsa/ecdsa.ts and bitgo/utils/tss/ecdsa/{base,ecdsa}.ts. A CRQC derives the private key from a public key that is already on chain after the first spend, so for a custodian the exposure is proportional to how much of its address set has ever been spent from.

23 reference(s) to ECDSA across 10 file(s), e.g. modules/sdk-core/src/account-lib/mpc/curves/index.ts:3. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:ECDSA:threshold_mpc
Data source	modules/sdk-core/src/account-lib/mpc/curves/index.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-core/src/account-lib/mpc/curves/index.ts:3 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · modules/sdk-core/src/account-lib/mpc/curves/index.ts · line 3 · SHA-256 38491911e06bbd81… · File 18ad0d545ab08eb4…

```
2 | import { Ed25519Curve } from './ed25519';
3 | import { Secp256k1Curve } from './secp256k1';
4 |
```

file_excerpt · modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts · line 3 · SHA-256 33ce4a2505533794… · File 0ff8c8b3a5edf4a2…

```
2 | import * as bigintCryptoUtils from 'bigint-crypto-utils';
3 | import { secp256k1 as secp } from '@noble/curves/secp256k1';
4 | import { createHash, Hash, randomBytes } from 'crypto';
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/ecdsa/base.ts · line 1 · SHA-256 7ccffd442cfe660a… · File c45c0d5c88b98b19…

```
1 | import { secp256k1 } from '@noble/curves/secp256k1';
2 |
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/ecdsa/base.ts · line 24 · SHA-256 71559810e4c22fbb… · File c45c0d5c88b98b19…

```
  23 |   async getBackupGpgPubKey(): Promise<BackupGpgKey> {
  24 |     return generateGPGKeyPair('secp256k1');
  25 |   }
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/ecdsa/base.ts · line 36 · SHA-256 420459093dd8b38c… · File c45c0d5c88b98b19…

```
  35 |     try {
  36 |       const point = secp256k1.ProjectivePoint.fromHex(pub);
  37 |       return point.toHex(false).slice(2);
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsa.ts · line 125 · SHA-256 4864758d7ef8f2d3… · File 9f97a990c329ce39…

```
  124 |     const userKeyShare = await MPC.keyShare(1, m, n);
  125 |     const userGpgKey = await generateGPGKeyPair('secp256k1');
  126 |     const backupKeyShare = await this.createBackupKeyShares();
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsa.ts · line 512 · SHA-256 3f6c3fc157a60476… · File 9f97a990c329ce39…

```
  511 |     })) as string;
  512 |     const userGpgKey = await generateGPGKeyPair('secp256k1');
  513 |     const privateShareProof = await createShareProof(userGpgKey.privateKey,
signingKey.nShares[bitgoIndex].u, 'ecdsa');
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaMPCv2.ts · line 83 · SHA-256 fc8b2023bf795af6… · File d9e80c98d4c22d63…

```
   82 |     const { userSession, backupSession } = this.getUserAndBackupSession(2, 3,
params.retrofit);
   83 |     const userGpgKey = await generateGPGKeyPair('secp256k1');
   84 |     const backupGpgKey = await generateGPGKeyPair('secp256k1');
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaMPCv2.ts · line 84 · SHA-256 8a0d0af5f75f20e0… · File d9e80c98d4c22d63…

```
   83 |     const userGpgKey = await generateGPGKeyPair('secp256k1');
   84 |     const backupGpgKey = await generateGPGKeyPair('secp256k1');
   85 |
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaMPCv2.ts · line 957 · SHA-256 cd9aa3730eb195a0… · File d9e80c98d4c22d63…

```
  956 |     let serializedTxHex: string | undefined;
  957 |     const userGpgKey = await generateGPGKeyPair('secp256k1');
  958 |     const bitgoGpgPubKey = await this.pickBitgoPubGpgKeyForSigning(true, params.reqId,
txRequest.enterpriseId);
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11789 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Paillier: threshold signing and MPC protocols (14 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** 42 references across 14 files, and the file list is the point: besides the protocol (account-lib/mpc/tss/ecdsa/{ecdsa,types}.ts) it reaches bitgo/keychain/{iKeychains,keychains}.ts, i.e. the keychain representation. Paillier material that is persisted is a harvest-now-decrypt-later item, not only a future-signing one. As with the other two custodian libraries, this is the row a post-quantum signature standard does not fix: Paillier underpins the threshold protocol, NIST's 2024 standards are single-signer, and the multi-party threshold effort has not produced one.

42 reference(s) to Paillier across 14 file(s), e.g. modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts:1. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:Paillier:threshold_mpc
Data source	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts:1 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts · line 1 · SHA-256 f36315a0a75eeaa4… · File 0ff8c8b3a5edf4a2…

```
1 | import * as paillierBigint from 'paillier-bigint';
2 | import * as bigintCryptoUtils from 'bigint-crypto-utils';
```

file_excerpt · modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts · line 99 · SHA-256 21295d6efabe896a… · File 0ff8c8b3a5edf4a2…

```
 98 |     // Generate additively homomorphic encryption key.
 99 |     let paillierKeyPair: paillierBigint.KeyPair;
100 |     if (!sync) {
```

file_excerpt · modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts · line 101 · SHA-256 b830135795327cff… · File 0ff8c8b3a5edf4a2…

```
100 |     if (!sync) {
101 |       paillierKeyPair = await paillierBigint.generateRandomKeys(minModulusBitLength, true);
102 |     } else {
```

file_excerpt · modules/sdk-core/src/account-lib/mpc/tss/ecdsa/types.ts · line 45 · SHA-256 45f7cf1478161923… · File abbd0202db16b8bf…

```
 44 |   c?: number; // number of shares
 45 |   l: string; // lambda value for paillier secret key
 46 |   m: string; // mu value for paillier secret key
```

file_excerpt · modules/sdk-core/src/account-lib/mpc/tss/ecdsa/types.ts · line 46 · SHA-256 0b79553c282b3756… · File abbd0202db16b8bf…

```
 45 |   l: string; // lambda value for paillier secret key
 46 |   m: string; // mu value for paillier secret key
 47 |   u: string; // shamir share of secret
```

file_excerpt · modules/sdk-core/src/account-lib/mpc/tss/ecdsa/types.ts · line 49 · SHA-256 d4eb6bbadd8e0ad8… · File abbd0202db16b8bf…

```
 48 |   uu: string; // unsplit shamir share of secret
 49 |   n: string; // n => (p . q) where p and q are the two random prime numbers chosen for
paillier encryption
 50 |   y: string;
```

file_excerpt · modules/sdk-core/src/bitgo/keychain/iKeychains.ts · line 237 · SHA-256 20ebcb726e72f496… · File 4139bff667e57bd1…

```
236 |   privateShareProof?: string;
237 |   paillierPublicKey?: string;
238 |   n?: string;
```

file_excerpt · modules/sdk-core/src/bitgo/keychain/keychains.ts · line 707 · SHA-256 4fd4e1b55f1725db… · File f71f4d7e57a5a248…

```
706 |     assert(bitgoToUserShare.vssProof);
707 |     assert(bitgoToUserShare.paillierPublicKey);
708 |     const bitgoToBackupShare = key.keyShares.find(
```

file_excerpt · modules/sdk-core/src/bitgo/keychain/keychains.ts · line 713 · SHA-256 0a12f2aa4c92d767… · File f71f4d7e57a5a248…

```
712 |     assert(bitgoToBackupShare.vssProof);
713 |     assert(bitgoToBackupShare.paillierPublicKey);
714 |
```

file_excerpt · modules/sdk-core/src/bitgo/keychain/keychains.ts · line 730 · SHA-256 feb54b0e71dcaf53… · File f71f4d7e57a5a248…

```
729 |                 privateShare: bitgoToUserShare.privateShare,
730 |                 paillierPublicKey: bitgoToUserShare.paillierPublicKey,
731 |                 vssProof: bitgoToUserShare.vssProof,
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11790 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Threshold ECDSA: threshold signing and MPC protocols (16 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** The largest row here — 215 references across 16 files — spanning account-lib/mpc/util.ts, bitgo/baseCoin/iBaseCoin.ts, bitgo/keychain/keychains.ts, bitgo/pendingApproval/pendingApproval.ts and the tss modules. It is correct by the product's purpose, and the spread through pending approvals and base coin interfaces shows why a migration is not confined to a crypto module: the threshold scheme's identity is baked into the SDK's public types, so changing it is an API-compatibility exercise across every integrating customer.

215 reference(s) to Threshold ECDSA across 16 file(s), e.g. modules/sdk-core/src/account-lib/mpc/util.ts:9. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:Threshold ECDSA:threshold_mpc
Data source	modules/sdk-core/src/account-lib/mpc/util.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-core/src/account-lib/mpc/util.ts:9 and inspect the quoted lines; the evidence lists
further files.
```

EVIDENCE

file_excerpt · modules/sdk-core/src/account-lib/mpc/util.ts · line 9 · SHA-256 9ecc65ea442d3f9c... · File d19b84bce55532d2...

```
    8 |   getPaillierPublicKey,
    9 |   DklsUtils,
   10 |   DklsTypes,
```

file_excerpt · modules/sdk-core/src/account-lib/mpc/util.ts · line 10 · SHA-256 75a62dec55df8a84... · File d19b84bce55532d2...

```
    9 |   DklsUtils,
   10 |   DklsTypes,
   11 | } from '@bitgo/sdk-lib-mpc';
```

file_excerpt · modules/sdk-core/src/account-lib/mpc/util.ts · line 19 · SHA-256 4b60f0e06656eabd... · File d19b84bce55532d2...

```
   18 | export function combineRound4DklsDsgMessages(
   19 |   round4DsgMessages: DklsTypes.SerializedBroadcastMessage[]
   20 | ): DklsTypes.SerializedDklsSignature {
```

file_excerpt · modules/sdk-core/src/bitgo/baseCoin/iBaseCoin.ts · line 9 · SHA-256 8317b0ca4f13e823... · File e61ed7c35599547a...

```
    8 | import { InitiateRecoveryOptions } from '../recovery';
    9 | import { EcdsaMPCv2Utils, EcdsaUtils } from '../utils/tss/ecdsa';
   10 | import EddsaUtils, { EddsaMPCv2Utils, PrebuildTransactionWithIntentOptions, TxRequest } from
'../utils/tss/eddsa';
```

file_excerpt · modules/sdk-core/src/bitgo/baseCoin/iBaseCoin.ts · line 377 · SHA-256 eb4bca49663f7f64... · File e61ed7c35599547a...

```
  376 |   walletData: WalletData;
  377 |   tssUtils: EcdsaUtils | EcdsaMPCv2Utils | EddsaUtils | EddsaMPCv2Utils |
RedpallasMPCv2Utils | undefined;
  378 |   [index: string]: unknown;
```

file_excerpt · modules/sdk-core/src/bitgo/keychain/keychains.ts · line 585 · SHA-256 3ee0fa76d6389e29... · File f71f4d7e57a5a248...

```
  584 |           ? ECDSAUtils.EcdsaVrfMPCv2Utils
  585 |           : ECDSAUtils.EcdsaMPCv2Utils
  586 |         : ECDSAUtils.EcdsaUtils;
```

file_excerpt · modules/sdk-core/src/bitgo/pendingApproval/pendingApproval.ts · line 25 · SHA-256 9595750d1a4a75c8... · File 3aa72e8fb06c0537...

```
   24 | import EddsaUtils, { EddsaMPCv2Utils } from '../utils/tss/eddsa';
   25 | import { EcdsaMPCv2Utils, EcdsaUtils } from '../utils/tss/ecdsa';
   26 | import { KeyShare as EcdsaKeyShare } from '../utils/tss/ecdsa/types';
```

file_excerpt · modules/sdk-core/src/bitgo/pendingApproval/pendingApproval.ts · line 58 · SHA-256 850c61ef35b95c66... · File 3aa72e8fb06c0537...

```
   57 |         if (this.wallet?.multisigTypeVersion() === 'MPCv2') {
   58 |           this.tssUtils = new EcdsaMPCv2Utils(this.bitgo, this.baseCoin, wallet);
   59 |         } else {
```

file_excerpt · modules/sdk-core/src/bitgo/tss/ecdsa/ecdsaMPCv2.ts · line 3 · SHA-256 93c900d8725293b4... · File 41179ffae02587f5...

```
2 | import openpgp from 'openpgp';
3 | import { DklsComms, DklsTypes } from '@bitgo/sdk-lib-mpc';
4 | import {
```

file_excerpt · modules/sdk-core/src/bitgo/tss/ecdsa/ecdsaMPCv2.ts · line 31 · SHA-256 23c08f39f71f63f5... · File 41179ffae02587f5...

```
30 | export async function getSignatureShareRoundOne(
31 |   round1Message: DklsTypes.DeserializedBroadcastMessage,
32 |   userGpgKey: openpgp.SerializedKeyPair<string>,
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11794 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable X25519: threshold signing and MPC protocols (2 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts and redpallas-mps/util.ts — the multi-party signing helpers for EdDSA and for RedPallas, the Zcash Orchard signature scheme. Worth noting for its own sake: a custodian supporting Zcash shielded signing carries the same discrete-log exposure there as everywhere else, and Orchard's own post-quantum future is the subject of Shielded Labs' Epoch rather than of this SDK.

4 reference(s) to X25519 across 2 file(s), e.g. modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts:3. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:X25519:threshold_mpc
Data source	modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts:3 and inspect the quoted lines; the evidence
lists further files.
```

EVIDENCE

file_excerpt · modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts · line 3 · SHA-256 3ea9cc6dfb31dbd5... · File b1f4fb0519c4b379...

```
  2 | import assert from 'assert';
  3 | import { x25519 } from '@noble/curves/ed25519';
  4 | import { DKG } from './dkg';
```

file_excerpt · modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts · line 20 · SHA-256 97dfa62e104f0cf3... · File b1f4fb0519c4b379...

```
 19 |   const privKey = seed ? seed.subarray(0, 32) : crypto.randomBytes(32);
 20 |   const pubKey = Buffer.from(x25519.getPublicKey(privKey));
 21 |   return { privKey: Buffer.from(privKey), pubKey };
```

file_excerpt · modules/sdk-lib-mpc/src/tss/redpallas-mps/util.ts · line 3 · SHA-256 a8787f3fa5393c76... · File 6ad2437c5cc62033...

```
  2 | import assert from 'assert';
  3 | import { x25519 } from '@noble/curves/ed25519';
  4 | import { RedPallasDKG } from './dkg';
```

file_excerpt · modules/sdk-lib-mpc/src/tss/redpallas-mps/util.ts · line 11 · SHA-256 186c6b01e10accbd... · File 6ad2437c5cc62033...

```
 10 |   const privKey = seed ? seed.subarray(0, 32) : crypto.randomBytes(32);
 11 |   const pubKey = Buffer.from(x25519.getPublicKey(privKey));
 12 |   return { privKey: Buffer.from(privKey), pubKey };
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11791 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: role not determined (2 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Role resolved, and it is a storage role rather than a signing one, which changes its priority. modules/sdk-core/src/bitgo/utils/opengpgUtils.ts wraps OpenPGP: it fetches BitGo's GPG public key (getBitgoGpgPubKey), encrypts and signs messages, and verifies share proofs over 'eddsa' | 'ecdsa' (verifyShareProof, verifySharedDataProof). That is how TSS key shares and their proofs are protected in transit and at rest, so harvest-now-decrypt-later applies: anything captured today is readable once a CRQC exists. The second anchor, bitgo/bip32util.ts, is BIP-32 key handling over @bitgo/secp256k1. Fixable today with a hybrid KEM around the share wrapping, independently of any signature migration.

4 reference(s) to ECDSA across 2 file(s), e.g. modules/sdk-core/src/bitgo/bip32util.ts:1. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:ECDSA:general
Data source	modules/sdk-core/src/bitgo/bip32util.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	40.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-core/src/bitgo/bip32util.ts:1 and inspect the quoted lines; the evidence lists
further files.
```

EVIDENCE

file_excerpt · modules/sdk-core/src/bitgo/bip32util.ts · line 1 · SHA-256 24d2a5062b7afe21… · File 96addf7021fa3ae1…

```
  1 | import { bip32utils } from '@bitgo/secp256k1';
  2 |
```

file_excerpt · modules/sdk-core/src/bitgo/utils/opengpgUtils.ts · line 17 · SHA-256 d15aa1888a51c6a9… · File e7caf153a36965e6…

```
 16 | import * as _ from 'lodash';
 17 | import { ecc as secp256k1 } from '@bitgo/secp256k1';
 18 | import { BitGoBase } from '../bitgoBase';
```

file_excerpt · modules/sdk-core/src/bitgo/utils/opengpgUtils.ts · line 116 · SHA-256 6d2a1b4250c65aea… · File e7caf153a36965e6…

```
 115 |     const decodedUValueProof =
Buffer.from(proofSubkeys.keyPacket.publicParams['Q']).toString('hex');
 116 |     const rawUValueProof = secp256k1.pointFromScalar(Buffer.from(uValue, 'hex'), false);
```

```
  117 |     return rawUValueProof !== null && decodedUValueProof ===
Buffer.from(rawUValueProof).toString('hex');
```

file_excerpt · modules/sdk-core/src/bitgo/utils/opengpgUtils.ts · line 254 · SHA-256 b065ed5bbec58a12... · File e7caf153a36965e6...

```
  253 |     oid.write = () => new Uint8Array(Buffer.from('052b8104000a', 'hex'));
  254 |     Q = secp256k1.pointFromScalar(new Uint8Array(Buffer.from(uValue, 'hex')), false);
  255 |   }
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11793 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable RedDSA: threshold signing and MPC protocols (2 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

2 reference(s) to RedDSA across 2 file(s), e.g. modules/sdk-core/src/bitgo/utils/tss/baseTSSUtils.ts:192. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:RedDSA:threshold_mpc
Data source	modules/sdk-core/src/bitgo/utils/tss/baseTSSUtils.ts
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:37 UTC
Priority score	35.6 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open modules/sdk-core/src/bitgo/utils/tss/baseTSSUtils.ts:192 and inspect the quoted lines; the
evidence lists further files.
```

EVIDENCE

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/baseTSSUtils.ts · line 192 · SHA-256 8a0cce9e39ef211f... · File c468bb1d3aadc97c...

```
  191 |       if (!this.bitgoRedpallasMpcv2PublicGpgKey) {
  192 |         throw new Error("Failed to get Bitgo's RedPallas MPCv2 gpg key");
  193 |       }
```

file_excerpt · modules/sdk-core/src/bitgo/utils/tss/redpallas/redpallasMPCv2KeyGenSender.ts · line 38 · SHA-256 effe122996112a7e... · File b283f6323c6833b1...

```
   37 |       .post(bitgo.url('/mpc/generatekey', 2))
   38 |       .send({ enterprise, safeId, type: KeyGenTypeEnum.MPCv2, curveType:
KeyCurveEnum.RedPallas, round, payload })
   39 |       .result();
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11795 · DEP-NO-PQC-LIBRARY

Info

No post-quantum capable cryptographic library detected

Insufficient data | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Accurate: across the three audited manifests (sdk-core, sdk-lib-mpc and the root package.json) the declared cryptography is @noble/curves, secp256k1 and tweetnacl, with nothing PQC-capable. In the JavaScript ecosystem, unlike on the Cortex-M targets in this portfolio, ML-KEM implementations are available (@noble/post-quantum among them), so for the key-wrapping rows above — OpenPGP share protection and the RSA-OAEP trading channel — a hybrid is a dependency choice today rather than a research problem. That makes this gap cheaper to close here than anywhere else in the custody slice, and it should be sequenced first.

Across 3 dependency manifests, QSI found crypto libraries (@noble/curves, secp256k1, tweetnacl) but none from its PQC-capable list.

Affected asset	dependencies
Data source	dependency manifests
Detection method	Dependency name matching.
Analyzer	dependencies 1.3.0 · 2026-11-04 18:37 UTC
Priority score	15.0 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

Readiness indicator only: PQC support may come from the platform (OpenSSL, OS, cloud KMS) rather than application dependencies.

REPRODUCTION STEPS

```
Review the dependency manifests listed in the evidence.
```

EVIDENCE

dependency · modules/sdk-core/package.json, modules/sdk-lib-mpc/package.json, package.json · SHA-256 5944c1b32f86f836…

```
Crypto libraries: @noble/curves, secp256k1, tweetnacl
```

LIMITATIONS OF THE CHECK

Absence of evidence is not evidence of absence: system libraries and transitive dependencies may not be listed.

RECOMMENDATION

Confirm where PQC support will come from (TLS terminator, KMS/HSM, library upgrade).

RETEST PROCEDURE

Re-scan after upgrades.

FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM); FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA)

Evidence

ID	Type	Data source	SHA-256
QSI-2026-11792	file_excerpt	modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts (line 33)	af9a7b3e6c70a3aedb3d7e3540657d072b9c668f9beea155d30c
QSI-2026-11792	file_excerpt	modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts (line 57)	bdce29586696e5a9765ad932c44cd0061fafd031a66d7044ad3b
QSI-2026-11786	file_excerpt	modules/sdk-core/src/account-lib/baseCoin/ed25519KeyPair.ts (line 16)	e8482eb972e2150ffaee797f8bcbb601d3cee29c49194d08b2e0
QSI-2026-11786	file_excerpt	modules/sdk-core/src/account-lib/baseCoin/index.ts (line 6)	af80a0b2e1e69e6516ad5f68f3db29a1e4fb950130eb0841ff6b
QSI-2026-11787	file_excerpt	modules/sdk-core/src/account-lib/baseCoin/secp256k1ExtendedKeyPair.ts (line 1)	6ddb812639dd12560936fdf7514b9ed108dea92bfcaa3192747f
QSI-2026-11787	file_excerpt	modules/sdk-core/src/account-lib/util/crypto.ts (line 2)	1b12201a01dddcd4c00bab3fe26bfa1f3844b33926ff56e3aa62
QSI-2026-11787	file_excerpt	modules/sdk-core/src/bitgo/ecdh.ts (line 16)	a1915605806bdb5915f32a17b9818b0f6f68d349788fbefb793e
QSI-2026-11787	file_excerpt	modules/sdk-core/src/bitgo/ecdh.ts (line 60)	7e080a30d6ce1730ece549b089d1f3a0639267ffd9ee8526456
QSI-2026-11787	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/addressVerification.ts (line 60)	796a023c4b3818bdff833f76ff14cf3951ca115eb7e08a833ba4
QSI-2026-11787	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/addressVerification.ts (line 90)	eaa6e881ef2e6ee89be000415903efa0191f3475f6cb8ade6d82
QSI-2026-11787	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/addressVerification.ts (line 94)	5fe68310e693027a1bdd79437677dcdba6e5656778789584e5d5
QSI-2026-11787	file_excerpt	modules/sdk-core/src/bitgo/utils/util.ts (line 196)	088bf5b0ec4051f2c76c55aa427efde7da6af74d629ecaf28cb2
QSI-2026-11787	file_excerpt	modules/sdk-core/src/bitgo/wallet/iWallet.ts (line 1403)	fea5f8cfea54429321c5f69fa72fc980f5260ba9004815988987
QSI-2026-11787	file_excerpt	modules/sdk-core/src/bitgo/wallet/wallet.ts (line 2517)	286358dd24445d05404c70a4485224af82c3240ab24190f4879e
QSI-2026-11788	file_excerpt	modules/sdk-core/src/account-lib/mpc/curves/index.ts (line 3)	38491911e06bbd81f86534e478f3f967811ac49f6ff724add23d

QSI-2026-11788	file_excerpt	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts (line 3)	33ce4a250553379449e6d8cd89cb89a0823ab8bf05b9505a6378
QSI-2026-11788	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/ecdsa/base.ts (line 1)	7ccffd442cfe660a7ceb11e8b2f709191a5fb41bf0b4612318dc
QSI-2026-11788	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/ecdsa/base.ts (line 24)	71559810e4c22fbb9359e84ec23b3fc6047de6e6374ba97286ef
QSI-2026-11788	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/ecdsa/base.ts (line 36)	420459093dd8b38c05054e2099e19afa8634737cb9a6f4c0f821
QSI-2026-11788	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsa.ts (line 125)	4864758d7ef8f2d39fc521d27d15d069f5af1c320a2eba9ae7d9
QSI-2026-11788	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsa.ts (line 512)	3f6c3fc157a60476ac0c2ce7bfbbdfda2ad2859c08d464cc3171
QSI-2026-11788	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaMPCv2.ts (line 83)	fc8b2023bf795af665da49334f471fe3e5f4077942eaa8b8d673
QSI-2026-11788	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaMPCv2.ts (line 84)	8a0d0af5f75f20e0c85edd068fd65251c02029154afcb7000c4e
QSI-2026-11788	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaMPCv2.ts (line 957)	cd9aa3730eb195a061809e54ca482bb224d15a657e485b640ec0
QSI-2026-11789	file_excerpt	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts (line 1)	f36315a0a75eeaa4119d9c7d953185879c7ce6f8cdf7a90f2e66
QSI-2026-11789	file_excerpt	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts (line 99)	21295d6efabe896a93732ccba672aa3b47cea5b44a118074ed20
QSI-2026-11789	file_excerpt	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts (line 101)	b830135795327cff07d5406c6dfb2830fe83071d265f2f63d116
QSI-2026-11789	file_excerpt	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/types.ts (line 45)	45f7cf14781619238472d456e85ef6011bcd9d01442e6bc26064
QSI-2026-11789	file_excerpt	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/types.ts (line 46)	0b79553c282b3756c51e93004b13b29d064b5bfbe21cef1b8776
QSI-2026-11789	file_excerpt	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/types.ts (line 49)	d4eb6bbadd8e0ad86375caec102086c5117c5495ad7143ad2471
QSI-2026-11789	file_excerpt	modules/sdk-core/src/bitgo/keychain/iKeychains.ts (line 237)	20ebcb726e72f496aaa3f34ba30c5820c5e540d211dc71aa42d4
QSI-2026-11789	file_excerpt	modules/sdk-core/src/bitgo/keychain/keychains.ts (line 707)	4fd4e1b55f1725db5a1b5972027c87d987d42279cd4d093b0554

QSI-2026-11789	file_excerpt	modules/sdk-core/src/bitgo/keychain/keychains.ts (line 713)	0a12f2aa4c92d767798132cfef166a90bcf5c43663d2a823a3b1
QSI-2026-11789	file_excerpt	modules/sdk-core/src/bitgo/keychain/keychains.ts (line 730)	feb54b0e71dcaf531493771d7e84516d2a8f4340c264cef95b19
QSI-2026-11790	file_excerpt	modules/sdk-core/src/account-lib/mpc/util.ts (line 9)	9ecc65ea442d3f9cc949c4c27780792b63d9b227160bd3a50390
QSI-2026-11790	file_excerpt	modules/sdk-core/src/account-lib/mpc/util.ts (line 10)	75a62dec55df8a8405500582c098c0c26a013d7eac00fc1fe752
QSI-2026-11790	file_excerpt	modules/sdk-core/src/account-lib/mpc/util.ts (line 19)	4b60f0e06656eabd89eddb6dc9be363b0c94a07c2fd35e6ead76
QSI-2026-11790	file_excerpt	modules/sdk-core/src/bitgo/baseCoin/iBaseCoin.ts (line 9)	8317b0ca4f13e823102fb6ae49badcc4828be8a0347ba4f5b886
QSI-2026-11790	file_excerpt	modules/sdk-core/src/bitgo/baseCoin/iBaseCoin.ts (line 377)	eb4bca49663f7f640cd282919f1a445e0755a1536fd149c07186
QSI-2026-11790	file_excerpt	modules/sdk-core/src/bitgo/keychain/keychains.ts (line 585)	3ee0fa76d6389e290e46a2b971e37dd1cb2fa97f384e764f70f0
QSI-2026-11790	file_excerpt	modules/sdk-core/src/bitgo/pendingApproval/pendingApproval.ts (line 25)	9595750d1a4a75c8f42ba1ccc11c036ae9f871ab1845010cd0c5
QSI-2026-11790	file_excerpt	modules/sdk-core/src/bitgo/pendingApproval/pendingApproval.ts (line 58)	850c61ef35b95c66f0dd241a81abec40e7b37501b38d23e7d09c
QSI-2026-11790	file_excerpt	modules/sdk-core/src/bitgo/tss/ecdsa/ecdsaMPCv2.ts (line 3)	93c900d8725293b4393bfde2e6d44596c6a7126f79a1602c028f
QSI-2026-11790	file_excerpt	modules/sdk-core/src/bitgo/tss/ecdsa/ecdsaMPCv2.ts (line 31)	23c08f39f71f63f536b9982f90413e648fa2dd434f1a732d7291
QSI-2026-11794	file_excerpt	modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts (line 3)	3ea9cc6dfb31dbd50b0077646438f05084cd46e56039edb8d217
QSI-2026-11794	file_excerpt	modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts (line 20)	97dfa62e104f0cf320a19007e1fb9ed7b948bb5636e765511fdd
QSI-2026-11794	file_excerpt	modules/sdk-lib-mpc/src/tss/redpallas-mps/util.ts (line 3)	a8787f3fa5393c76bf6722126e120557397e05469cb4f7027926
QSI-2026-11794	file_excerpt	modules/sdk-lib-mpc/src/tss/redpallas-mps/util.ts (line 11)	186c6b01e10accbdc3539c7dfcf7765f1c7dca0d349bb3216bb1
QSI-2026-11791	file_excerpt	modules/sdk-core/src/bitgo/bip32util.ts (line 1)	24d2a5062b7afe21ee900b0935378d339afe6e4c184d07bc46ba
QSI-2026-11791	file_excerpt	modules/sdk-core/src/bitgo/utils/opengpgUtils.ts (line 17)	d15aa1888a51c6a930577236bcd602bf7b7fcef313a80831424c
QSI-2026-11791	file_excerpt	modules/sdk-core/src/bitgo/utils/opengpgUtils.ts (line 116)	6d2a1b4250c65aea50ed876f3685b5f89ca132e51a5d84822d5a
QSI-2026-11791	file_excerpt	modules/sdk-core/src/bitgo/utils/opengpgUtils.ts (line 254)	b065ed5bbec58a128bd36fb1574f9df5ec42c1b87887566389ba

QSI-2026-11793	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/baseTSSUtils.ts (line 192)	8a0cce9e39ef211f2145a16a77b56f4b06f6d2ec95d3a00235a6
QSI-2026-11793	file_excerpt	modules/sdk-core/src/bitgo/utils/tss/redpallas/redpallasMPCv2KeyGenSender.ts (line 38)	effe122996112a7e09eda11f83f8fe0f6c3f8841bb838fdfc315
QSI-2026-11795	dependency	modules/sdk-core/package.json, modules/sdk-lib-mpc/package.json, package.json	5944c1b32f86f8360d871035e5113a02613db7978e2f95438e67

Post-quantum assessment

	Key establishment (x = how long data must stay confidential; harvest now, decrypt later)	Signatures (x = how long signatures/ keys must stay trusted)
Data/key lifetime x	10	10
Migration time y	5	5
Assumed years to CRQC z	8 (early)	8
x + y − z	7 → migration would finish too late under this scenario	7 → migration would finish too late under this scenario

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography

- Classical only — key wrapping and data encryption at rest: RSA (1 file(s))
- Classical only — threshold signing and MPC protocols: ECDSA, Paillier, RedDSA, Threshold ECDSA, X25519 (38 file(s))

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

What the quantum-vulnerable algorithms do

Role	Algorithms (files)	ID	First step
key wrapping and data encryption at rest	RSA (1)	QSI-2026-11792	Wrap keys with ML-KEM or a hybrid KEM (X25519 + ML-KEM-768) and re-wrap the stored keys; data that may already have been copied needs its own assessment
threshold signing and MPC protocols	Threshold ECDSA (16), Paillier (14), ECDSA (10), RedDSA (2), X25519 (2)	QSI-2026-11788, QSI-2026-11789, QSI-2026-11790, QSI-2026-11793, QSI-2026-11794	Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long
transaction and message signing	ECDSA (10), Ed25519 (2)	QSI-2026-11786, QSI-2026-11787	Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them

role not determined	ECDSA (2)	QSI-2026-11791	Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Quantum status	Count
reduced margin (Grover), approved	9
Grover: acceptable to NIST, below ECCG/ANSSI guidance	9
n/a	7
vulnerable (Shor)	68

Detailed findings: QSI-2026-11792, QSI-2026-11786, QSI-2026-11787, QSI-2026-11788, QSI-2026-11789, QSI-2026-11790, QSI-2026-11794, QSI-2026-11791, QSI-2026-11793

PQC readiness questionnaire

No readiness questionnaire was completed for this project.

Quantum algorithms relevant to this inventory

Quantum algorithm	Effect	Inventoried algorithms (rows)	What it means for this audit
Shor's algorithm — factoring Q1: quantum computation offline, classical access to the victim	breaks	Paillier (14), RSA (3)	Breaks RSA at every key length: a longer modulus only raises the cost polynomially. Plan migration to ML-KEM (or a hybrid) for key establishment and ML-DSA / SLH-DSA for signatures; start with keys protecting data that must stay confidential for years (harvest now, decrypt later) and with long-lived signatures.
Shor's algorithm — elliptic-curve discrete logarithms Q1: quantum computation offline, classical access to the victim	breaks	ECDH (1), ECDSA (23), Ed25519 (7), RedDSA (2), Threshold ECDSA (16), X25519 (2)	Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime.

Grover's algorithm — key search and preimages Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	AES-256 (2), HMAC (4), SHA-2 (12)	Not a break. 256-bit keys (AES-256, ChaCha20) keep a comfortable margin; AES-128 meets NIST category 1, while some authorities ask for 256-bit keys in quantum-resistant contexts (see the AES-128 entry). Prefer 256-bit keys for long-lived data; 256-bit hashes need no change for preimage resistance.
Brassard–Høyer–Tapp — collision search Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	SHA-2 (12)	256-bit hashes (SHA-256, SHA3-256) remain adequate for collision resistance; 160-bit hashes are already weak classically (SHA-1 collisions are practical). Use 384/512-bit hashes where NIST categories 4–5 are required.
Simon's algorithm — superposition attacks on symmetric constructions Q2: superposition queries to the victim's keyed primitive	only with superposition queries	GCM mode (2)	No action for ordinary deployments of AES-GCM or CBC-MAC — the attack model does not apply. Record it as a model assumption; see the offline variant for Even–Mansour-type designs.

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.
- keys — error (not checked): modules/sdk-core/src/bitgo/trading/network/encrypt.ts line 19: public key could not be parsed (ValueError)
- keys — error (not checked): modules/sdk-core/src/bitgo/trading/network/encrypt.ts line 45: public key could not be parsed (ValueError)
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 8: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 12: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 17: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 18: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 21: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 22: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 27: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 28: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 31: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 32: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 43: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 44: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 47: OpenPGP key block could not be parsed
- keys — error (not checked): modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts line 48: OpenPGP key block could not be parsed

Remediation plan

QSI-2026-11792 · P3 · 90 days

short-term risk reduction

Add RSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11786 · P3 · 90 days

short-term risk reduction

Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11787 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.

Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11788 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11789 · P3 · 90 days

short-term risk reduction

Add Paillier usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11790 · P3 · 90 days

short-term risk reduction

Add Threshold ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.

Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11794 · P3 · 90 days

short-term risk reduction

Add X25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11791 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11793 · P3 · 90 days

short-term risk reduction

Add RedDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.

Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11792 · P4 · 90 days root-cause fix

Plan migration of RSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-11786 · P4 · 90 days root-cause fix

Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-11787 · P4 · 90 days root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.

Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-11788 · P4 · 90 days root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-11789 · P4 · 90 days root-cause fix

Plan migration of Paillier to No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) or a hybrid scheme in use.

QSI-2026-11790 · P4 · 90 days root-cause fix

Plan migration of Threshold ECDSA to The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.

Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design or a hybrid scheme in use.

QSI-2026-11794 · P4 · 90 days

root-cause fix

Plan migration of X25519 to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH or a hybrid scheme in use.

QSI-2026-11791 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-11793 · P4 · 90 days

root-cause fix

Plan migration of RedDSA to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows or a hybrid scheme in use.

Priorities for 30, 60 and 90 days

30 days

Priority	ID	Action	Type
None			

60 days

Priority	ID	Action	Type
None			

90 days

Priority	ID	Action	Type
P3	QSI-2026-11792	Add RSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11786	Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11787	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11788	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11789	Add Paillier usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11790	Add Threshold ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11794	Add X25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11791	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11793	Add RedDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P4	QSI-2026-11792	Plan migration of RSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix

P4	QSI-2026-11786	Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11787	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11788	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11789	Plan migration of Paillier to No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11790	Plan migration of Threshold ECDSA to The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11794	Plan migration of X25519 to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11791	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11793	Plan migration of RedDSA to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix

Verification after remediation

ID	Retest procedure
QSI-2026-11792	Re-run the inventory after refactoring.
QSI-2026-11786	Re-run the inventory after refactoring.
QSI-2026-11787	Re-run the inventory after refactoring.
QSI-2026-11788	Re-run the inventory after refactoring.
QSI-2026-11789	Re-run the inventory after refactoring.
QSI-2026-11790	Re-run the inventory after refactoring.
QSI-2026-11794	Re-run the inventory after refactoring.
QSI-2026-11791	Re-run the inventory after refactoring.
QSI-2026-11793	Re-run the inventory after refactoring.
QSI-2026-11795	Re-scan after upgrades.

Tools used

Tool	Version
blockchain	1.0.0
certificates	1.2.0
chain_map	1.0.0
configs	1.1.0
dependencies	1.3.0
inventory_document	1.1.0
solidity	1.3.0
source_code	1.4.0
tls_code	1.1.0
tls_live	1.0.0
qsi-engine	1.3.0
OSV.dev API	v1

Sources and normative documents

Reviewed: 2026-10-13

1. A. Chailloux, M. Naya-Plasencia, A. Schrottenloher — An Efficient Quantum Collision Search Algorithm and Implications on Symmetric Cryptography (ASIACRYPT 2017; IACR ePrint 2017/847) — IACR ePrint. https://eprint.iacr.org/2017/847 (peer-reviewed article: Õ(2^{2n/5}) time, O(n) qubits, Õ(2^{n/5}) classical memory)
2. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
3. C. Gidney — How to factor 2048 bit RSA integers with less than a million noisy qubits (arXiv:2505.15917) — arXiv. https://arxiv.org/abs/2505.15917 (preprint (2025-05-21): < 1 million noisy qubits, < 1 week, 0.1% gate error, 1 µs surface-code cycle)
4. C. Gidney, M. Ekerå — How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits (Quantum 5, 433) — Quantum. https://doi.org/10.22331/q-2021-04-15-433 (peer-reviewed article (2021): 10^-3 gate error, 1 µs cycle, 10 µs reaction time)
5. C. Zalka — Grover's quantum searching algorithm is optimal (Phys. Rev. A 60, 2746, 1999; arXiv:quant-ph/9711070) — APS. https://doi.org/10.1103/PhysRevA.60.2746 (peer-reviewed article: optimal for any success probability; parallel search gains no more than splitting the space)
6. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
7. D. J. Bernstein — Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? (2009) — cr.yp.to. https://cr.yp.to/hash/collisioncost-20090823.pdf (paper dated 2009-08-23)
8. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
9. D. R. Simon — On the Power of Quantum Computation (SIAM J. Comput. 26, 1997) — SIAM. https://doi.org/10.1137/S0097539796298637 (peer-reviewed article)
10. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
11. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
12. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
13. G. Brassard, P. Høyer, A. Tapp — Quantum Algorithm for the Collision Problem (LATIN '98; arXiv:quant-ph/9705002) — Springer / arXiv. https://arxiv.org/abs/quant-ph/9705002 (peer-reviewed article: collisions of r-to-one functions in O((N/r)^{1/3}) expected evaluations)
14. H. Kuwakado, M. Morii — Quantum distinguisher between the 3-round Feistel cipher and the random permutation (IEEE ISIT 2010) — IEEE. https://ieeexplore.ieee.org/document/5513654 (peer-reviewed conference paper)
15. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
16. L. K. Grover — A fast quantum mechanical algorithm for database search (STOC 1996) — ACM. https://doi.org/10.1145/237814.237866 (peer-reviewed article)
17. M. Amy et al. — Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3 (SAC 2016; arXiv:1603.09383) — Springer / arXiv. https://arxiv.org/abs/1603.09383 (peer-reviewed article: SHA-256 about 2^153.8 surface-code cycles on 2^12.6 logical qubits; SHA3-256 about 2^146.5 cycles on 2^20)
18. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
19. M. Kaplan, G. Leurent, A. Leverrier, M. Naya-Plasencia — Breaking Symmetric Cryptosystems using Quantum Period Finding (CRYPTO 2016; arXiv:1602.05973) — Springer / arXiv. https://arxiv.org/abs/1602.05973 (peer-reviewed article: superposition-query forgeries on CBC-MAC, PMAC, GMAC, GCM, OCB)
20. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)
21. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)

22. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/final (final (2026-10-27; the initial public draft of November 2024 is obsoleted). Tables 2 and 4: ECDSA, EdDSA and RSA signatures, finite-field and elliptic-curve DH and MQV, and RSA key establishment are deprecated after 2030 and disallowed after 2035 at every key size — the draft deprecated only the 112-bit level. A composite (hybrid) key establishment stays FIPS 140-validatable while one input comes from an approved scheme. Repeats the federal deadlines of OMB M-26-15: post-quantum key establishment by 2030-12-31 and signatures by 2031-12-31 for high value assets and high impact systems)
23. NIST — Post-Quantum Cryptography Standardization: Evaluation Criteria, Security (Call for Proposals, Section 4.A.5) — NIST. https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization/Evaluation-Criteria/Security-(Evaluation-Criteria) (security categories: AES-128 key search 2^170/MAXDEPTH quantum gates, AES-192 2^233/MAXDEPTH, AES-256 2^298/MAXDEPTH; MAXDEPTH 2^40–2^96)
24. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)
25. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)
26. S. Jaques, M. Naehrig, M. Roetteler, F. Virdia — Implementing Grover oracles for quantum key search on AES and LowMC (EUROCRYPT 2020; arXiv:1910.01700) — Springer / arXiv. https://arxiv.org/abs/1910.01700 (peer-reviewed article: lower cost estimates for the AES key searches that define the NIST categories)

Appendices

A. Cryptographic inventory

Algorithm	Parameter	Quantum status	Classical status	Count	Locations	Examples
AES-256	AES-256	reduced margin (Grover), approved	acceptable	2	2	modules/sdk-core/src/bitgo/trading/network/decrypt-aes-gcm.ts:25; modules/sdk-core/src/bitgo/trading/network/encrypt-aes-gcm.ts:18
HMAC		reduced margin (Grover), approved	acceptable	4	4	modules/sdk-core/src/bitgo/webauthn/deriveEnterpriseSalt.ts:22; modules/sdk-lib-mpc/src/curves/ed25519Bip32HdTree.ts:60; modules/sdk-lib-mpc/src/curves/secp256k1Bip32HdTree.ts:59
SHA-2	SHA-512	reduced margin (Grover), approved	acceptable	3	3	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts:95; modules/sdk-core/src/account-lib/mpc/tss/eddsa/eddsa.ts:86; modules/sdk-core/src/bitgo/safe/derivableEd25519Pub.ts:78
SHA-2	SHA-256	Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable	9	9	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts:178; modules/sdk-core/src/bitgo/baseCoin/baseCoin.ts:533; modules/sdk-lib-mpc/src/hashCommitment.ts:29
Crypto library @noble/curves (noble-curves)	1.8.1	n/a	acceptable	2	2	modules/sdk-core/package.json; modules/sdk-lib-mpc/package.json
Crypto library secp256k1 (secp256k1 bindings)	5.0.1	n/a	acceptable	2	2	modules/sdk-core/package.json; modules/sdk-lib-mpc/package.json
Crypto library tweetnacl (TweetNaCl)	^1.0.3	n/a	acceptable	1	1	modules/sdk-core/package.json
GCM mode		n/a	acceptable	2	2	modules/sdk-core/src/bitgo/trading/network/decrypt-aes-gcm.ts; modules/sdk-core/src/bitgo/trading/network/encrypt-aes-gcm.ts
ECDH		vulnerable (Shor)	acceptable	1	1	modules/sdk-core/src/bitgo/ecdh.ts

ECDSA	secp256k1	vulnerable (Shor)	acceptable	19	19	modules/sdk-core/src/account-lib/baseCoin/secp256k1ExtendedKeyPair.ts:1; modules/sdk-core/src/account-lib/mpc/curves/index.ts:3; modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts:3
ECDSA		vulnerable (Shor)	acceptable	4	4	modules/sdk-core/src/bitgo/tss/ecdsa/ecdsa.ts; modules/sdk-core/src/bitgo/utils/util.ts:196; modules/sdk-core/src/bitgo/wallet/iWallet.ts:1403
Ed25519	Ed25519	vulnerable (Shor)	acceptable	4	4	modules/sdk-core/src/account-lib/baseCoin/ed25519KeyPair.ts:16; modules/sdk-core/src/account-lib/baseCoin/index.ts:6; modules/sdk-core/src/account-lib/mpc/curves/ed25519.ts
Ed25519		vulnerable (Shor)	acceptable	3	3	modules/sdk-core/src/account-lib/mpc/tss/eddsa/eddsa.ts; modules/sdk-core/src/bitgo/tss/eddsa/eddsa.ts; modules/sdk-core/src/bitgo/utils/tss/eddsa/eddsa.ts
Paillier		vulnerable (Shor)	acceptable	14	14	modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts:1; modules/sdk-core/src/account-lib/mpc/tss/ecdsa/types.ts:45; modules/sdk-core/src/bitgo/keychain/iKeychains.ts:237
RSA		vulnerable (Shor)	acceptable	3	3	modules/sdk-core/src/bitgo/trading/network/decrypt-rsa.ts; modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts:33; modules/sdk-core/src/bitgo/trading/network/encrypt-rsa.ts
RedDSA	RedPallas	vulnerable (Shor)	acceptable	2	2	modules/sdk-core/src/bitgo/utils/tss/baseTSSUtils.ts:192; modules/sdk-core/src/bitgo/utils/tss/redpallas/redpallasMPCv2KeyGenSender.ts:38
Threshold ECDSA		vulnerable (Shor)	acceptable	16	16	modules/sdk-core/src/account-lib/mpc/util.ts:9; modules/sdk-core/src/bitgo/baseCoin/iBaseCoin.ts:9; modules/sdk-core/src/bitgo/keychain/keychains.ts:585
X25519	X25519	vulnerable (Shor)	acceptable	2	2	modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts:3; modules/sdk-lib-mpc/src/tss/redpallas-mps/util.ts:3

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

B. JSON

qsi.report/1 · SHA-256 38bce0925d7c971c44f19b16d5f05ed81e6d3158320e060b824044b1cb6ac506

C. Input files

File	Size	SHA-256
modules/sdk-core/package.json	2251	0e81526b78ab89713bfa71986890080c68d9665984db27d5d79a801cd2b178
modules/sdk-core/src/account-lib/baseCoin/baseKeyPair.ts	604	29c889672a037cd997f86e8987cb1cf0a1f990da0ee2c3ef419e9eb9a898d5
modules/sdk-core/src/account-lib/baseCoin/baseTransaction.ts	2939	ae5e15c4ee2a43cde794e05dcba0a62769a44c8b49a13220ccbd3f78a60644
modules/sdk-core/src/account-lib/baseCoin/baseTransactionBuilder.ts	7771	138975f7856c995e8f66e79d00dc78c581acc3abf838054d234d64d29e036b
modules/sdk-core/src/account-lib/baseCoin/baseTransactionBuilderFactory.ts	1025	89092049114ebc6899b4ec026eb2ae4d0e7c1b16038ee023d2784d0f9530b0
modules/sdk-core/src/account-lib/baseCoin/baseUtils.ts	1471	60f1976fa671c2bedd4bab63c6b5d6441153fa7cde081e711875683365ed9d
modules/sdk-core/src/account-lib/baseCoin/ed25519KeyPair.ts	4639	7ea2c42fb769971f98e82d81eba11e44699712db8da3f789624d8e00682d8e
modules/sdk-core/src/account-lib/baseCoin/enum.ts	5940	1e3debb6cbbc6cbfcf8f28dc01306a53cd08a2da231ab71ecc8fa5b11e7581
modules/sdk-core/src/account-lib/baseCoin/errors.ts	2323	5511b3ee7754255d456d05521af080a837e61aa24f4f01aa8adcb6c56c7c66
modules/sdk-core/src/account-lib/baseCoin/iface.ts	4656	1816708b7cad47d4a1aa40f153e5afd2157c86305c871dc3a5758559ca2a43
modules/sdk-core/src/account-lib/baseCoin/index.ts	574	ba3540f3b0cc2c17e53275356f6f3799bd54e19cbd476af8e043934495418b
modules/sdk-core/src/account-lib/baseCoin/messages/baseMessage.ts	5340	681ab3df2ff41442d04b256624c1e47e2fe5331917e8af61030d5850e372bc
modules/sdk-core/src/account-lib/baseCoin/messages/baseMessageBuilder.ts	4667	f1ee4d5151a354b3cdd59584c9369a323fc90c3cc239bfbc9856f245b52016
modules/sdk-core/src/account-lib/baseCoin/messages/baseMessageBuilderFactory.ts	2105	149e325c4114384499eece1dd6b51e71fdc8f51c703886237ac1847d3de44d
modules/sdk-core/src/account-lib/baseCoin/messages/iface.ts	5715	8e2c03b7ac40eb5e02a780240484de4ffd9e051b1660db90be6d6d7ed26920
modules/sdk-core/src/account-lib/baseCoin/messages/index.ts	165	d78a0140316223aee32c9d0b15727e3a7fd3bf008082d12758f5c6c1bba532
modules/sdk-core/src/account-lib/baseCoin/messages/simple/index.ts	73	fd5cdabd9263a8a77db8e777cdfeff21089a96b2d639faff37e2d1df17be21
modules/sdk-core/src/account-lib/baseCoin/messages/simple/simpleMessage.ts	633	019a899aa14894d3008acc3dcf5a02c07391d753c387798c1b5d0eb5f04ed9
modules/sdk-core/src/account-lib/baseCoin/messages/simple/simpleMessageBuilder.ts	893	bb5b463f599a42637af511f6077b7c8ad6b0c6c982f3b68ebbbf19a64bf1eb
modules/sdk-core/src/account-lib/baseCoin/secp256k1ExtendedKeyPair.ts	3641	411521d568ce8dee7adcfb0f2dfc5ae9355e875bf414e0e3b3930642b427e9

modules/sdk-core/src/account-lib/index.ts	237	6e3caebca4d9afb7d0a301a557473cbc891ec28aa60a01eb4df8754cff7e55
modules/sdk-core/src/account-lib/mpc/curves/baseCurve.ts	218	f79121fef63d4dcfc9216d1c93d4f7a51a4db7548dc1be29ed593dae06f020
modules/sdk-core/src/account-lib/mpc/curves/ed25519.ts	116	7923f40a701c308b20f7c385974ac459a5c94b8ab0de661f600bd566da6785
modules/sdk-core/src/account-lib/mpc/curves/index.ts	185	18ad0d545ab08eb42d73fb1613270c80e33e075ae3a656cfe9f713a79b71ca
modules/sdk-core/src/account-lib/mpc/curves/secp256k1.ts	120	33d33573ce9965491ec5f43ef35db0e22313742f18704e08b1d3fc9a5a2eb4
modules/sdk-core/src/account-lib/mpc/index.ts	413	ccaee9747186cc95a6a67526f9580409689ab8a7986af8ab6db7c421cd57f1
modules/sdk-core/src/account-lib/mpc/shamir.ts	127	77ce77238f2ac7783a0017bd0802f24661266a0bcb6efd17826ad5f6ab8355
modules/sdk-core/src/account-lib/mpc/tss/ecdsa/ecdsa.ts	59234	0ff8c8b3a5edf4a2283cb4ddecc7b9add5c4a0798f3f5a93fc5ef421662f24
modules/sdk-core/src/account-lib/mpc/tss/ecdsa/index.ts	131	446d0ed1a5467113b904aaee1d4177816e0b1769de26af992ce079843d9a5a
modules/sdk-core/src/account-lib/mpc/tss/ecdsa/rangeproof.ts	637	5d40fd184e77b4cf826e830872e24565392e07de7962127ee6fd48c1ed85c4
modules/sdk-core/src/account-lib/mpc/tss/ecdsa/types.ts	7573	abbd0202db16b8bf05a4ae2c48c3a7ae276502ff69017126738fffb843ddfd
modules/sdk-core/src/account-lib/mpc/tss/eddsa/eddsa.ts	13204	2bc585fc8935a9796b919761ae4ec08290b9004b17ac86f9539b24ed40c843
modules/sdk-core/src/account-lib/mpc/tss/eddsa/index.ts	111	b07a12211b6e4d928d34c875aa336743ebaf95f8cba35a075abcc73628f858
modules/sdk-core/src/account-lib/mpc/tss/eddsa/types.ts	1147	9e673bc0c94020104ec503d907781696aa5f09813b217cd791ab182247760b
modules/sdk-core/src/account-lib/mpc/tss/index.ts	282	cf12abf5ff49c9d1b8cbce54f6d8b830389049a913cf0b30f3308f93a9dac4
modules/sdk-core/src/account-lib/mpc/util.ts	1739	d19b84bce55532d269ce2919e4321ac2ad9883f08f7cc2057d8b1e4b994f2c
modules/sdk-core/src/account-lib/staking/index.ts	25	7bb6dfa42f1353cfcd5a8dd77a509471436b3068e1a947deb309b39cc3c011
modules/sdk-core/src/account-lib/staking/utils.ts	3165	212000ef6efddc9723ac4c5a5b3f7a62e9710c09686943a237c32f645a9018
modules/sdk-core/src/account-lib/util/crypto.ts	6790	d619311bd82ddd745a68372d39045458c17cdcb8c2e20fee968589eac34b8f
modules/sdk-core/src/api/bip32path.ts	1495	35926cf456631166daab38a23a430fc3616b64052f68cceeb46c237c181eab
modules/sdk-core/src/api/index.ts	54	2acb6b79354aebbbfa708c281ba8f19e0d3029db9c6d679ddd493801e2b360
modules/sdk-core/src/api/types.ts	1774	92100b5007d237788d9497d9cbd70b37f56533eb664370eab75fff0e7c1aca

modules/sdk-core/src/bitgo/address-book/address-book.ts	7382	124fbd430a8e8a324107505774b84f055a1684a535e144fbb9185b27467f7e
modules/sdk-core/src/bitgo/address-book/index.ts	57	f4246d43ae715a11e63bc14cbe41852dc75b54802ed50d3f4e88b4d912b3ea
modules/sdk-core/src/bitgo/address-book/types.ts	6451	b91796b852907434c9506c19e075d8d6c84c83015316cacd47e2ddaaa28d68
modules/sdk-core/src/bitgo/baseCoin/baseCoin.ts	22961	29c5a6560db0b0bea3767c2a4dbe23134f3b73e1f2829ed27d97cff8085e64
modules/sdk-core/src/bitgo/baseCoin/iBaseCoin.ts	27979	e61ed7c35599547af0546d7c8ef961a62a6da724fcc4a194b96836e5e6d86d
modules/sdk-core/src/bitgo/baseCoin/index.ts	57	fa7828312d84b7d311e6e4f2fbbb131bf80d41a51fb741c668cde40204471d
modules/sdk-core/src/bitgo/bip32util.ts	154	96addf7021fa3ae1ac80f6f9d8983d9026ee8287e339500064e13c6a0dcd45
modules/sdk-core/src/bitgo/bitcoin.ts	1018	738f22f37cbdb67a23263f084cd74fd9839c772710c0f9d27cfb1199984eeb
modules/sdk-core/src/bitgo/bitgoBase.ts	2370	ed3e380ed8650c4118e800c7b05f8d172829f4ee60b66705531b6ea2824af2
modules/sdk-core/src/bitgo/coinFactory.ts	3079	88f2c8c2f17eb58d28685cecb6a543c235568bcea992c734b4b497db93d9aa
modules/sdk-core/src/bitgo/config.ts	1692	ccda17269f9a069b0666ce71a3e7eb88a3b87baa5c4b502ffc070dd23f7932
modules/sdk-core/src/bitgo/defi/defiVault.ts	18449	c9b128c48908dca82532842838bf491b53aa53985696caa02431b4af2399c8
modules/sdk-core/src/bitgo/defi/iDefiVault.ts	3178	eac49305699959419e54eaccf6e5c374d1d2f227f3c7d8f6d8a654c3ae44f8
modules/sdk-core/src/bitgo/defi/index.ts	99	09f486e8618c7e0600b00d94a3349cac864d6d5d8877703704640595215dc3
modules/sdk-core/src/bitgo/ecdh.ts	3201	817c83e398ed1f0ca1c7d3aa2e33652a26af92206ff36a88262921d6d5a3f8
modules/sdk-core/src/bitgo/enterprise/enterprise.ts	8838	1df4a133e5880a41a31e776f96e7ba0e1282054b620a99fade1a6cda500a96
modules/sdk-core/src/bitgo/enterprise/enterprises.ts	3336	8be08737b49e3a9a337d976966905037243fda09bc5327a02fb877705431d1
modules/sdk-core/src/bitgo/enterprise/iEnterprise.ts	1777	507b5739a569fb134123b9a1af27c094a3e838a40b3047ef8b5e80c4b1baa9
modules/sdk-core/src/bitgo/enterprise/iEnterprises.ts	479	a95dfd988cbdf7ca7552d1d3c78923666bccb1b0ee0fbb41ddb14e85357fa6
modules/sdk-core/src/bitgo/enterprise/index.ts	124	b3edd21907680a84bc7bbc6c72c37b77e93ca0fd4f82f4080b7b6a2254d087
modules/sdk-core/src/bitgo/environments.ts	25728	9005bbdee6d5528f204f8e8f5423e488a862be6c31091fba2f8aea1f2a57cb
modules/sdk-core/src/bitgo/errors.ts	16558	d71100f60665a6cc1915f03584697fb95c74291ddb72b59f15d75191b7f5bb
modules/sdk-core/src/bitgo/evm/evmUtils.ts	2402	a17cdfef1f43a5796e9e06aaddd68ef2e22062b0190bbe30a378a1deeae0fb
modules/sdk-core/src/bitgo/index.ts	1007	37ed83a4463e2eba3e8d72832443b7b0b6533e22352e6411753b036d9b0f34
modules/sdk-core/src/bitgo/inscriptionBuilder/iInscriptionBuilder.ts	2116	4e5318a54e68c5eeef913bbc72f5dd69452126bc4db83e52045015367939a6

modules/sdk-core/src/bitgo/inscriptionBuilder/index.ts	39	2362353e2d9faf8016a5b46a3da76823cf48156b3018dd371e4f80f1e62396
modules/sdk-core/src/bitgo/internal/index.ts	55	d4da9754445ea9eefe81c37e29c40fc5541fc5716c01676896575f1ad7dfea
modules/sdk-core/src/bitgo/internal/internal.ts	1125	a3411f08ae3502388a551af3f54c27c316cc02b53884144e61581e5687546b
modules/sdk-core/src/bitgo/internal/keycard.ts	13067	7adfed79dada8c588ac49a280ddd26d918c5caf2b54e64a65eb9259b81c33a
modules/sdk-core/src/bitgo/keychain/decryptKeychain.ts	1139	901cb0d413b7c9ed869e47e278e6e992458b2b904e5f7067ed70f1e844ec7c
modules/sdk-core/src/bitgo/keychain/iKeychains.ts	12634	4139bff667e57bd1d17d25bac93a02eca94fda1937956d0b583335a9717e4b
modules/sdk-core/src/bitgo/keychain/index.ts	126	cc3a975cf343854328fb7549a99ff97876cbd5dbdc836d373db0be4aca4976
modules/sdk-core/src/bitgo/keychain/keychains.ts	34301	f71f4d7e57a5a2487de3c789c293c16938585ea38fbb21f5618ddd94816f4c
modules/sdk-core/src/bitgo/keychain/ovcJsonCodec.ts	1803	8e8fa46b85b847a411622f0719416076c92255c5de9ae8c0cc9509e625b663
modules/sdk-core/src/bitgo/legacyBitcoin.ts	2361	e081d3330add02aa246b2a30c6c6f27f68718f2ecc199d05457e7cf9c62ee2
modules/sdk-core/src/bitgo/lightning/lightningWalletUtil.ts	1173	8add3010e35ccc0c2197be9bf48bd702e26795f9926a1ebbd0bd298c188ffa
modules/sdk-core/src/bitgo/market/iMarkets.ts	510	8339e981a787f62feff9a471c82f8b696cc80b26580eca0c81a18b4a488c02
modules/sdk-core/src/bitgo/market/index.ts	55	59a22f4bfc55490c51d999c805802f25f015ec21d83c11aa5db20a48a7d6fd
modules/sdk-core/src/bitgo/market/markets.ts	1907	d77d86686983a7413ec0d242dd08e1001bf8ea4525a9f98145e46e85fcd1f9
modules/sdk-core/src/bitgo/pendingApproval/iPendingApproval.ts	2388	2192b952d74833e7f9b3838d508853547280024a2ebe42b59c8a64eb76f67b
modules/sdk-core/src/bitgo/pendingApproval/iPendingApprovals.ts	496	683335d31f39b462175ed7d9a089700360614e857d01c1ae358ef50447ace9
modules/sdk-core/src/bitgo/pendingApproval/index.ts	144	af43e2fbefcb987495af3885475bc20fec590234df2ca53afd1891d38f1052
modules/sdk-core/src/bitgo/pendingApproval/pendingApproval.ts	19278	3aa72e8fb06c0537296bcdafdd871834450acc7df15e8c9647fa71365dc37d
modules/sdk-core/src/bitgo/pendingApproval/pendingApprovals.ts	2405	82bbb4ab4eca6571e203fa7bc111d52fe781f4531127b264e7312d30775a8c
modules/sdk-core/src/bitgo/proofs/WalletProofs.ts	1440	19e634ff8287b2de9be2ee73311f9142d06c3282e6c3086cda6ee96ddbd5ee
modules/sdk-core/src/bitgo/proofs/index.ts	47	7f64b2ef50d68c6484e5c900839065c85ebcaf6ce7e3ea9c49767bdc7925b3
modules/sdk-core/src/bitgo/proofs/types.ts	1534	386335d6a47d7acc672d632cbb63c360212a3b9537e456f2fd42b572eb4ace

modules/sdk-core/src/bitgo/recovery/index.ts	28	8bf9eb184fe55d122cbda02300633f9ccf8f740c8d519959052bdfc958355f
modules/sdk-core/src/bitgo/recovery/initiate.ts	4704	adb1956fcf999c2dbd18a7334ddc23162c8e0e1a20d3b0ff12331b49b926d1
modules/sdk-core/src/bitgo/safe/derivableEd25519Pub.ts	5962	a6377e50c494f961754db31d1e52e5e1b2921f845ab458fafc27be03d52d79
modules/sdk-core/src/bitgo/safe/iSafe.ts	4561	9afc11bb89c6f44092dcd1ceede7aca2eb9c65840af091d3b408425a49b83d
modules/sdk-core/src/bitgo/safe/iSafes.ts	3211	8964b909438fc3fca4df1008d4c8a24358bc3fd70eced69fd94c4e7ab6177f
modules/sdk-core/src/bitgo/safe/index.ts	205	602a24dad185602a86c8c4edb0b3f235ab3d1aadc9353e973edcf64cf02559
modules/sdk-core/src/bitgo/safe/recovery.ts	10250	0f844eba89b7071e4633cb0ab7b591da870a51bea77f3708b2bdd42b011e83
modules/sdk-core/src/bitgo/safe/rootCoin.ts	1788	fc3e2f52403c8db03089fd97557bef112d58118fc268ff3f854e79ddf4c36a
modules/sdk-core/src/bitgo/safe/safe.ts	16328	b3817db1f61cca040a775781ce942bef92dbdaee3dc97fa7b8d7e2e4d22268
modules/sdk-core/src/bitgo/safe/safes.ts	12285	d1ab02bf5ae75215d5e3b6c5ee879d8c9575e868aab66bcb3cffab2d7b4d43
modules/sdk-core/src/bitgo/staking/goStakingInterfaces.ts	3437	8924ceccb762d4aa32fac978219c9e646e110d652765dee94126ae41c12e7d
modules/sdk-core/src/bitgo/staking/goStakingWallet.ts	5352	edeae7a4ee3702bca2bfed33b4177ea50db6522c245c96f1a43e16320f7a4a
modules/sdk-core/src/bitgo/staking/iGoStakingWallet.ts	967	2225918a88ada40950ae6110e12fe0e734ac76ad0838c9f46bc1467c3ae29b
modules/sdk-core/src/bitgo/staking/iStakingWallet.ts	8800	7560669d366c5c00a147cce896a8798390e7a1f76565e152e81f3f5545e606
modules/sdk-core/src/bitgo/staking/index.ts	177	b6e7ae0f6ef90ccf4d41a35b26c35a98d368237167abdac6576c138b5542f6
modules/sdk-core/src/bitgo/staking/stakingWallet.ts	18432	7999b82e532dda8499270793b270ab12ea3c725b0812143bc2122c70999e69
modules/sdk-core/src/bitgo/trading/iTradingAccount.ts	881	b6ef7374061bbcdf4ec74c226d62c38bd685545153e1af17263b4f43545e4a
modules/sdk-core/src/bitgo/trading/index.ts	96	c65d776d945dc1ca6561626381e2aaa82e69c6039213cd1ec922d7f453c37e
modules/sdk-core/src/bitgo/trading/network/decrypt-aes-gcm.ts	971	060407b619c7815776529d55632f95ef3a33bc6e1609a053f8337807aa41fd
modules/sdk-core/src/bitgo/trading/network/decrypt-rsa.ts	599	b121c99f8d152e5708dde23d743e55b0d5453797bf4fd54632a8aae1fe1794
modules/sdk-core/src/bitgo/trading/network/decrypt.ts	964	7fc07fd1c8e8047cf8031c412b3b07a017f078cf8f34600fc6d6891bae6a67
modules/sdk-core/src/bitgo/trading/network/encrypt-aes-gcm.ts	794	4ed2935bca9dd7ccb44cba68818f3a4c65978758976df5f8f0402db52a388a
modules/sdk-core/src/bitgo/trading/network/encrypt-rsa-browser.ts	2202	77177724e5cbd0642d89f37da00c7ffa240bbdc893ace55b90de0269db6ab5
modules/sdk-core/src/bitgo/trading/network/encrypt-rsa.ts	564	8f11ea1f91d45d8f2cdc5d0396908103adc3d7516975b81d4689810faf8ecc

modules/sdk-core/src/bitgo/trading/network/encrypt.ts	2391	b0fe3509c49b28cf616ce924e9aa067a772f8f49f8c55f3595801b56b6926e
modules/sdk-core/src/bitgo/trading/network/index.ts	106	2543a7554fa447db36e54ea66aa375f04cf6110da3c7a363d43cb11f28c6b3
modules/sdk-core/src/bitgo/trading/network/network.ts	7538	9d40389854964cec5cc585bc935ff0c99b464bce7b803301c3dbe631ea7739
modules/sdk-core/src/bitgo/trading/network/types.ts	9109	159287a1956e5f7fc04217c3504e01b3d906bd3c0a11745a75f5cc47c5e65b
modules/sdk-core/src/bitgo/trading/network/utils.ts	1407	e53db00695b8c31c8c9a86c61d8431e79ec2f9c7dbe2b2cb3353be20bc6936
modules/sdk-core/src/bitgo/trading/tradingAccount.ts	4400	57091835510abbd38679bfd93d7d778dc5ffe056277654a9caaf43e9c7d075
modules/sdk-core/src/bitgo/tss/bitgoPubKeys.ts	21832	43b47c9bf7d6382ae9f554d3428190951249bd29281840173008ed0fa418fe
modules/sdk-core/src/bitgo/tss/common.ts	12204	4483ab6f6957f1aa609f64b14527e8282cc1f0a31b0b475699a3c9587c1539
modules/sdk-core/src/bitgo/tss/ecdsa/ecdsa.ts	30723	277aa9d7cbcc3ed596eb14c3a548089d5e36a6f09e2fc5dd64915687fad3ad
modules/sdk-core/src/bitgo/tss/ecdsa/ecdsaMPCv2.ts	8031	41179ffae02587f5add47e01203f15820fe7271dd578b57d60911fedc685ce
modules/sdk-core/src/bitgo/tss/ecdsa/index.ts	160	7df43aa2c865d02b407a3fe4dfbd8a14b597f1c25f3ae08cf54665f3eb30b1
modules/sdk-core/src/bitgo/tss/ecdsa/types.ts	2233	21bfd54f395a989da0b06787c244ac33df9551ece6275568838201460c6d4d
modules/sdk-core/src/bitgo/tss/eddsa/eddsa.ts	15727	4fa8944c5d7820be021bf8f3fcabbe97b17256e097590f37f6f15eb5c00dbc
modules/sdk-core/src/bitgo/tss/eddsa/eddsaMPCv2.ts	5097	44a5e37bcc7763fc165531e67f567976342c2a16be343d7e6bc6d12ccab422
modules/sdk-core/src/bitgo/tss/eddsa/index.ts	490	f15f3c57d9633b1cc96f60f41f03bfff8bfc7ace73f1a2fe1eb984c0f94126
modules/sdk-core/src/bitgo/tss/eddsa/types.ts	1164	1842f092d2761e1202a4ff112cabf20d0c0d833d55227f497cd9c4357074ee
modules/sdk-core/src/bitgo/tss/index.ts	754	a48b3920f373c5f3691fdf591e6477934a34eb8b862d37bea3c6a60a4c7eb4
modules/sdk-core/src/bitgo/tss/redpallas/redpallasMPCv2.ts	14488	bd04074eb5a00cf44330faa1c0ff8ca04d00eb518273328e018213a7f73c49
modules/sdk-core/src/bitgo/tss/types.ts	855	2d2b44ae4fffab7937e5fce77aa3b97b7e601a6c5371d9e3e7e5687977dfa3
modules/sdk-core/src/bitgo/types.ts	198	651a757b4950d14f779350e604641b5b25567f22bbc372408de10591b1fc6d
modules/sdk-core/src/bitgo/utils/abstractUtxoCoinUtil.ts	3177	9c387f630a4950fde2c000fcde592512908bd7e95419b031486bf86139dcfc
modules/sdk-core/src/bitgo/utils/codecProps.ts	529	d7458269556ed44bd534f00781e591cbd803b8e4651d611ec421f15ed8e748

modules/sdk-core/src/bitgo/utils/codecs.ts	1502	3f3877bd219d5350af1a8c3537edf774441856789d6b9b5b81522ebd92e947
modules/sdk-core/src/bitgo/utils/decode.ts	1835	3349a9c1033483cfab1b99e20bed946d21285d5a0816fcf40e9c2d457f6605
modules/sdk-core/src/bitgo/utils/index.ts	443	72ca3ef5bb32ad15e06176183ec701441ed54be649c62859a5ac865fd1ca8d
modules/sdk-core/src/bitgo/utils/messageTypes.ts	1421	46c69ef47ac6fbb28a8c99ffe0f7b72b86856924c82301be59565edff2cb1a
modules/sdk-core/src/bitgo/utils/mpcUtils.ts	16547	26238d1c0106a74428a32c2b6853e9b4354b1e6ae13856b7cdfb117578c192
modules/sdk-core/src/bitgo/utils/notEmpty.ts	139	417a0a52e2abde7a8e9ed80c233cbe4eb2b3d386c69e07db9a474042ffcc22
modules/sdk-core/src/bitgo/utils/opengpgUtils.ts	15701	e7caf153a36965e6dbe00b2b3479cd7a145b9d33ec7df2bb61de8d1d4d75ed
modules/sdk-core/src/bitgo/utils/postWithCodec.ts	1303	dc64d9de774f56db5634dbdfbe4044e5a99f31a900ee203dd169ef642fcb7a
modules/sdk-core/src/bitgo/utils/promise-utils.ts	374	c951312973c57f624791e838042cf2712d6f695c38d879c1d75a3070cc0f75
modules/sdk-core/src/bitgo/utils/siws.ts	5053	8398d9910f6a4c3a7e9659d25e902a30418156d4d483ccfad275c8d5050f02
modules/sdk-core/src/bitgo/utils/transactionUtils.ts	883	643a3ca25fcfe330d55a7560fb873b571436ebb1ec79a957a0de8f5aee9a3f
modules/sdk-core/src/bitgo/utils/triple.ts	123	eafc8908dca504220220de30b0664776803d2a7732cffbde3a2553c8e7e747
modules/sdk-core/src/bitgo/utils/tss/addressVerification.ts	5627	84e20697851f775c288e660881f6ec37ab4264c2007f8689ebf578ff9aaba2
modules/sdk-core/src/bitgo/utils/tss/baseTSSUtils.ts	30712	c468bb1d3aadc97c5530a4860d88a4bd436ee8e6feed938e260d6d095261cf
modules/sdk-core/src/bitgo/utils/tss/baseTypes.ts	29966	5432317a9a26809f727ad8aa9124a1c416b975204b9605719c22b2bd941974
modules/sdk-core/src/bitgo/utils/tss/ecdsa/SMC/utils.ts	8509	cc3dcb12401501d966acdb28169e1e37c4cff0ee5a561711f35e6432e163b8
modules/sdk-core/src/bitgo/utils/tss/ecdsa/base.ts	2046	c45c0d5c88b98b193ba362f7f406b3e41bb58989fd61e6f4b45604fb35074c
modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsa.ts	48828	9f97a990c329ce39426a3dbcea0fb9f5df2ef57fc77ed37e5fe44c211cb836
modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaMPCv2.ts	70208	d9e80c98d4c22d6316f95f86e343c2273293a26fc5c8e553f90aa255c581d9
modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaMPCv2KeyGenSender.ts	2027	982fe3af61138ac34b330ba60d186c6d8b402c1a67cfab38a3aadd9864b816
modules/sdk-core/src/bitgo/utils/tss/ecdsa/ecdsaVrfMPCv2.ts	33983	80e6a71c0a822f360577fc5a4d068ea092e09e9e248854fe0d8b65261b9a94
modules/sdk-core/src/bitgo/utils/tss/ecdsa/index.ts	214	67980833f7f19a9ce647dec1caf5891bb658241a6ac7500ee84436a5d3f423
modules/sdk-core/src/bitgo/utils/tss/ecdsa/types.ts	1237	608ea833bbabef4d4afda2b87f49587e30ef1e4868988c080cfe7fee5b669d

modules/sdk-core/src/bitgo/utils/tss/ecdsa/typesMPCv2.ts	1236	c18066e3aa36916475077b8951803dc95da8e2bb534f7a3351aea29e1886cf
modules/sdk-core/src/bitgo/utils/tss/eddsa/SMC/utils.ts	6272	e3a2603f02e7e3025c5a390fb58498b3497fb85a0ad428660b6c24ec96ae64
modules/sdk-core/src/bitgo/utils/tss/eddsa/base.ts	1026	2f50147cc6ecce1360e38d37f0512bdf98e28b6b4abf38862e0cf50af52535
modules/sdk-core/src/bitgo/utils/tss/eddsa/eddsa.ts	32651	4ce44fd08fd895f80e2f5d1c339782935e9c293a875d9c8036269e6a4f0e4f
modules/sdk-core/src/bitgo/utils/tss/eddsa/eddsaMPCv2.ts	64088	dd74221115ad6491edb942d58d7a2a5bdcef09b75d1b242ccf879ce3ee77de
modules/sdk-core/src/bitgo/utils/tss/eddsa/eddsaMPCv2KeyGenSender.ts	1951	8b53400494421d62dcb8192cea4f431df3a324f335c3e2affab31549a0e801
modules/sdk-core/src/bitgo/utils/tss/eddsa/eddsaVrfMPCv2.ts	18822	67b3a9fc41936d8e611bb1c5f9d80b5cd0c90ba7efdcafa4b087f75d20c53b
modules/sdk-core/src/bitgo/utils/tss/eddsa/index.ts	621	6d3274234af6441563bbd2b6e681c9e61a98043925807e110790e2fb10866f
modules/sdk-core/src/bitgo/utils/tss/eddsa/types.ts	1291	c216252106b103bc04b4eb6490ec0f7fc85327330496bc8d2ec15aedbb3ecd
modules/sdk-core/src/bitgo/utils/tss/eddsa/typesEddsaMPCv2.ts	1059	5faf163d3018cf9260834fdc556d98523aea3e9de34697821d00396d2b8cc2
modules/sdk-core/src/bitgo/utils/tss/index.ts	828	8d517619ef1970a521fa2be511e36fe49a51424934cf9b478088d54b60bfda
modules/sdk-core/src/bitgo/utils/tss/keyShareEnvelope.ts	4820	e5d82a00691ff419e7c8619eeb9ca6f5f6c42ecf51e499fa30d04a0105e58a
modules/sdk-core/src/bitgo/utils/tss/keyVerification.ts	7318	b2dcc9564d71ee54d04860fb18536b6a616b604503c1dcfb626fbcd568926c
modules/sdk-core/src/bitgo/utils/tss/preHashedSignable.ts	1380	e7505190899d7f6d3c2abcceb92db7d5d73289011d6e88e4b950044a518f92
modules/sdk-core/src/bitgo/utils/tss/recipientUtils.ts	8680	cf2eca3543991e8ba7ecf944f439056ff4afa0db177a9db169cbc9d5f058d4
modules/sdk-core/src/bitgo/utils/tss/redpallas/SMC/utils.ts	6886	c229d3cb8214f9f6220ef2e1db352d757703de0c5406fe05b555016c6e9de8
modules/sdk-core/src/bitgo/utils/tss/redpallas/index.ts	109	5dda862dd4e7daec9cf4f539af50b625b63eb2afd4bc3e0aa5ce97c2bfb7e8
modules/sdk-core/src/bitgo/utils/tss/redpallas/redpallasMPCv2.ts	3083	13795b43eeb7a5a9516a5052f51faadd3e810ff061be5e56ef984a4423fd55
modules/sdk-core/src/bitgo/utils/tss/redpallas/redpallasMPCv2KeyGenSender.ts	1434	b283f6323c6833b1af527f24c15fb05b1fa426d2c8c9e190085cda5733568e
modules/sdk-core/src/bitgo/utils/tss/serializedTxHexVerify.ts	529	ee9f95c480d9eb02cbe3670fb4de60d8eda3bd6cd9b658ff9aa88a0ff9d811
modules/sdk-core/src/bitgo/utils/tss/signableConsistency.ts	807	bd49415b9e8342f7a0f1627fafca64b9c5299784c1e5877b4f00c3efbc471e

modules/sdk-core/src/bitgo/utils/txRequest.ts	1602	43152cfc9fdc4307ebdbfd3a10dc2c92d18d8f795ac6e77dbd49d274d5ea02
modules/sdk-core/src/bitgo/utils/util.ts	5234	ad33cca15d17c2905548311ffd094aa2a310d15b1914454241f5a184334f1a
modules/sdk-core/src/bitgo/utils/wallet.ts	356	8390bbef247b2f592d8e0033229f3a386ae6418c43e10ee236958537cc9427
modules/sdk-core/src/bitgo/v1Wallets.ts	6651	a280393b8275025484f1b8211e52395048675ffe613438bee0c8f7cde6b791
modules/sdk-core/src/bitgo/wallet/BuildParams.ts	6018	322eb0c814cfaef665fb51baf0c3c4494c950f05cc0cd3258be6230012491c
modules/sdk-core/src/bitgo/wallet/iWallet.ts	42977	d66cfa8b23933fbfdc036a49a5ec6cd88491480ceb8c9778ea5864775328bc
modules/sdk-core/src/bitgo/wallet/iWallets.ts	16166	3333a22818589f4be341c237ee607ca87b8d7085f892595979a292b9e5385c
modules/sdk-core/src/bitgo/wallet/index.ts	140	d3fc9da92d0ae3f03f13b1ba9b4f587eab7d98075d58d815ab9b82b95e892b
modules/sdk-core/src/bitgo/wallet/safeKeychain.ts	8933	2aba3d9f2debca43af5ca9cc6ceb4717e6300dd330f0f35b56904f179cf953
modules/sdk-core/src/bitgo/wallet/wallet.ts	219409	2cee1dfbd5165bde1c0df80f855b60d914dea73a5211a4e968eeace71cb4f9
modules/sdk-core/src/bitgo/wallet/wallets.ts	82521	20b0b3c91a3de429ead0b94def7d74cfdf2eef9101beb4e44325bb4200732c
modules/sdk-core/src/bitgo/walletUtil/iMessageProvider.ts	424	d0b2ae6fd43c5efb88c1ba3accc2485b2ea9b6620e2acea9e3ba5e65364ea8
modules/sdk-core/src/bitgo/walletUtil/index.ts	93	37cbee3ee2d52fd48f6414175dad7fe6dffcb1babe5c3bad0cef4b5cced91b
modules/sdk-core/src/bitgo/walletUtil/midnightMessageProvider.ts	2775	f33404ffb879151fe5496d9f0e4aeafe8ae76b500c2d1e03314aec2c84f4bc
modules/sdk-core/src/bitgo/walletUtil/signAccountBasedMidnightClaimMessages.ts	3416	94969e893285210b2ff0ee5c8a027ecc40f86b68e4bb01df22819e6ea02494
modules/sdk-core/src/bitgo/walletUtil/utxoMessageProof.ts	2058	1f7e5a922237cb14c1455e039bb4504977b3b052d87bd384394b39fb8229f9
modules/sdk-core/src/bitgo/webauthn/base64url.ts	990	75cd056f0f94e22af514678bb289514d29ba83cd885f39735f33c807cabf7b
modules/sdk-core/src/bitgo/webauthn/deriveEnterpriseSalt.ts	1112	52d6df5ce374d5451de868ade0dbd6f4226f435385c805901fb0fa65314e61
modules/sdk-core/src/bitgo/webauthn/derivePassword.ts	456	9c79403c2584664de5ae2714601e9ad3b876e9c45b2ee44a52bbf385c60f51
modules/sdk-core/src/bitgo/webauthn/index.ts	128	44d3e23988cc554090c865e043a078bfebd546cfcb48e4a92275d0164857e8
modules/sdk-core/src/bitgo/webauthn/types.ts	661	741ae75ae88b07145539a8f6e0d0147cbf80ed75b54d703e0747bf1f7dbc66
modules/sdk-core/src/bitgo/webhook/iWebhooks.ts	653	d4ec0046d622b5849e2980cd2c58247d53c07badbe350a0b2384e5f190d0f2
modules/sdk-core/src/bitgo/webhook/index.ts	57	b38b31cd0d2a33394106638287abcfa9a92a3211126c22cd1c072479dc3979

modules/sdk-core/src/bitgo/webhook/webhooks.ts	2583	886b0e6956266b1d68074f5045143552cee82bc6f57db0e8ffcb51aa84e894
modules/sdk-core/src/bitgojsError.ts	148	c52a0da37f3e3da6039848cc58ccd652a3b7624c6caf8f0629ec1726e4a4cf
modules/sdk-core/src/coins/fiat.ts	3725	89206e7b9d3f6e78817b3b2350ab6a951e17eca334f6a3b0a51839f856c80d
modules/sdk-core/src/coins/index.ts	124	73eeead977e3a5c07b064d3de45bd705922ce78104c8f981743f2e8bd5395f
modules/sdk-core/src/coins/ofc.ts	4047	c4c0e8ae90a93ad6593afe69a213ce9c4b30edaedbe46794f3e37ec4b556a7
modules/sdk-core/src/coins/ofcToken.ts	4863	e3d0cb5ae40c32eea80f5372322fa25d1c4b4bb70a4deea13ecf8b728da718
modules/sdk-core/src/coins/susd.ts	1788	8118fcc5829221021d2aeace91ec52d658f4ffef5bccdf0c81d774a1dd3417
modules/sdk-core/src/coins/tsusd.ts	316	1624831c405e833248137ae723cd36fdf1a709026b4949360cb8309c1bc5c4
modules/sdk-core/src/common.ts	2204	02f021d19076b9bdc57c2925993b8f1ab5cc21603d37f12d630ba24ba59319
modules/sdk-core/src/index.ts	1932	ceebb0b24dbed570f6e3e0604824ddc6a254fd37688f4b3cf331ea284d0f00
modules/sdk-core/src/lightning.ts	625	02ef9839f3f16c282e4f3e4af8b1d5a366e9037bcdd249e37fb805f277f409
modules/sdk-core/src/units.ts	2181	414b27df289a9e7614b23d2c83a4200d85f85ac6fb07338796031ca10aac25
modules/sdk-lib-mpc/package.json	2650	1c0994b03a6001629185656b76921028ab958f6a013c160aa1333d32b9e78f
modules/sdk-lib-mpc/src/curves/constant.ts	118	2ed0c884d409eb0da69b6d6240044647ace41f2b8083e428756bed2e936158
modules/sdk-lib-mpc/src/curves/ed25519.ts	3223	7b9024da9dc5d7881c775f62cc352232d1a9c6eed8420d77e633be9db1a94e
modules/sdk-lib-mpc/src/curves/ed25519Bip32HdTree.ts	3799	9ade9764263cc0b1bf04fbac82daea86f40b118716b8d94714c64cfd5142d4
modules/sdk-lib-mpc/src/curves/index.ts	211	f08909ab42e7d1b52466ede578622402845b7d41f45619546915dc6429a0d1
modules/sdk-lib-mpc/src/curves/secp256k1.ts	2059	2e8f795dfdc94c8589b8f58dda54a52923bae0a39a0e9f15eb0253e9f3d3bb
modules/sdk-lib-mpc/src/curves/secp256k1Bip32HdTree.ts	2422	974e47a6fb6f415b90dcc1aca01125e31101df6c911d5e90b00ea547fccafd
modules/sdk-lib-mpc/src/curves/types.ts	1953	308e44dddce383c689e3fc72ffd058eb7122e4a136b434f63ef9716b6c69aa
modules/sdk-lib-mpc/src/curves/util.ts	1447	cc96d8a63535fc99e3766d4cdc875211ee615912fefbc4774505fbfa855501
modules/sdk-lib-mpc/src/hashCommitment.ts	1475	d59a3bc33ad801a9b7ecafba6c62f09191d79a060403e50c5d4af9934446a0
modules/sdk-lib-mpc/src/index.ts	249	28640cdd64a277b0822d2b0c6a4fcd6f8301fe34ac0735af6a17de23574191
modules/sdk-lib-mpc/src/openssl/index.ts	27	36b406442e4dacaa64c3e4ae32859f5b4f8e853ef81d607dbe67c9c9c302ce
modules/sdk-lib-mpc/src/openssl/openssl.ts	1219	9d426081a8762c0bfcc62d5e4dc10df480f2d2a54070991ca8478d935e9169
modules/sdk-lib-mpc/src/safePrime.ts	613	1173367ae142d18e53fa7013e00393d769a759c62f315a2bd5a859432b6aa6
modules/sdk-lib-mpc/src/schnorrProof.ts	2507	a2b0dfa9874fbc8ca182fe7c2060b86764f0483843e614a6380a6c19580ed9
modules/sdk-lib-mpc/src/shamir/index.ts	51	263c93c7e6746d71967f7a5a433ebac963c24368d2381631faef6ad6d82674
modules/sdk-lib-mpc/src/shamir/shamir.ts	4643	de438bb75333fe68a63ff1686dfd40366f2e0190614e06dd87abd73be556dd
modules/sdk-lib-mpc/src/shamir/types.ts	80	52260b8eae9826f161fbfccec9b5fcd41c80344737a0b5f8c0838f90de6e82

modules/sdk-lib-mpc/src/tss/dkls-vrf/dkg.ts	11419	f36cea29eb1581d45a117d673161df55100e33bfe767bd9c125f9d5b91b313
modules/sdk-lib-mpc/src/tss/dkls-vrf/index.ts	115	40f2c968e9768de6036f2deb7328cfe874bd6624007d17cbb77f465d84e231
modules/sdk-lib-mpc/src/tss/dkls-vrf/types.ts	951	41dfefaa59378e293c833eb857a466765872edeb2971b1ed31203d19fdce78
modules/sdk-lib-mpc/src/tss/dkls-vrf/util.ts	9639	84267a8b39f33252189ea8631c47ae1cd8caaf5beee68c7c7cc910107c14db
modules/sdk-lib-mpc/src/tss/ecdsa-dkls/commsLayer.ts	9450	f3120211ab7f76a1724d1ede712b6b8437116a3cf352c8ed73a131b66e67e9
modules/sdk-lib-mpc/src/tss/ecdsa-dkls/derive.ts	14567	e38b4db24aa5e768c83b55b8079b0595dc943a466d3c4936e8d6eb324d6270
modules/sdk-lib-mpc/src/tss/ecdsa-dkls/dkg.ts	11631	2a7a8ee891944ab7412b7acc5d173668dd1cbd87b207a5772f6b8192719b5e
modules/sdk-lib-mpc/src/tss/ecdsa-dkls/dsg.ts	9105	58da2d5bf5408ba380ce563067d6734bb72ebb28cc3323e28f700c51d97390
modules/sdk-lib-mpc/src/tss/ecdsa-dkls/index.ts	223	98b18c4cf62904c596c914e9fa018cef74684a3f78206572f991583b996dd2
modules/sdk-lib-mpc/src/tss/ecdsa-dkls/types.ts	8625	3a4cd2dccf083f0d3bd2a730010d912d3a9041d51807a8c540b2b04fb9fe8e
modules/sdk-lib-mpc/src/tss/ecdsa-dkls/util.ts	10718	8f8f4d31813aaaa2b13ab34c183c26dc835dd47eb96289d241c0b423210add
modules/sdk-lib-mpc/src/tss/ecdsa/index.ts	230	e8299f41f710f5264e86c4c9c9cecc40e6a15adf56227738313f1cc9e3121b
modules/sdk-lib-mpc/src/tss/ecdsa/paillierproof.ts	2997	eaced94643ae69cb88d3d7c43b312d9d101ed3fdadb7fa8106c0332df0639c
modules/sdk-lib-mpc/src/tss/ecdsa/primes.ts	213282	d5b762b692d14b48d000077e07ff3dcb05d09a472cfaddfb32d80f87adcc21
modules/sdk-lib-mpc/src/tss/ecdsa/rangeproof.ts	16931	74bdd421eee91cf93385b3cf15bb56a1446a9565f8bd59f01c34ade578f6f4
modules/sdk-lib-mpc/src/tss/ecdsa/types.ts	6772	29c2c6ea40eb8912276fe022aa7c917b4fab66138cd17ea8872263465f9bfa
modules/sdk-lib-mpc/src/tss/ecdsa/zkVProof.ts	3284	306429197e0f2fb4010bdea45ef675f2713ea063d8ad5ddd07e2f114364b38
modules/sdk-lib-mpc/src/tss/eddsa-mps-vrf/dkg.ts	8730	6001c683b0db6c4f7176100331f0914192014136c219d4e96166b05492d490
modules/sdk-lib-mpc/src/tss/eddsa-mps-vrf/index.ts	112	099d5d99d96c537a948167668fa125cb8e7164a6bd1e0cc90f5b0ce55e691d
modules/sdk-lib-mpc/src/tss/eddsa-mps-vrf/types.ts	2676	6a9350f7bb5cad691e77c47c5c2ff21b0799dafd87a0a5087005842d4999d0
modules/sdk-lib-mpc/src/tss/eddsa-mps-vrf/util.ts	4215	9340006dee36f12d4d52d33a66ca64cdabaf63b7b57f45da1b4c2114a3a2a5
modules/sdk-lib-mpc/src/tss/eddsa-mps/commsLayer.ts	2568	9c59876a51ea5ee9e846406fde30fa5650fd0ba7158352f5f1f4fb11b7224a

modules/sdk-lib-mpc/src/tss/eddsa-mps/derive.ts	6105	00499722b20c0666f2f19d5606127cdfe93703e82f55110aa9acbdd3e259a7
modules/sdk-lib-mpc/src/tss/eddsa-mps/dkg.ts	11988	e9449d170d22a278e54de1944bd5e7bf132c960decc3118f37409025e42fad
modules/sdk-lib-mpc/src/tss/eddsa-mps/dsg.ts	13383	01a17b37d72926427fa566b39a13ae78e5720579b01b1add79d93356653047
modules/sdk-lib-mpc/src/tss/eddsa-mps/index.ts	279	2f295aa4383cfe86ab2566e820c16287029b7aa52ab0087b8865bc07991ff1
modules/sdk-lib-mpc/src/tss/eddsa-mps/types.ts	3589	4cd5035fd2dfd525c9b94883cc34638aec317496d6f7d704c05c2ecc7d050b
modules/sdk-lib-mpc/src/tss/eddsa-mps/util.ts	5200	b1f4fb0519c4b37924a268e3b3424329ec0f8d3e6839edcc540719923f12cc
modules/sdk-lib-mpc/src/tss/index.ts	178	1f58188875d8c9a5eec2d23d773be0587bd318b818430cd99c6be930348d1b
modules/sdk-lib-mpc/src/tss/redpallas-mps/commsLayer.ts	2774	aacb878883ed3d6b2213239bb2c0e412a5f66a2ea0641c31dd5010d6ac2230
modules/sdk-lib-mpc/src/tss/redpallas-mps/dkg.ts	10988	57df5a72e9b714ec8340baa27fcca79381a1b600d1cce1c29edbe174012421
modules/sdk-lib-mpc/src/tss/redpallas-mps/dsg.ts	15730	be2a9d287b8334d5e48446f68c80b9e20ee9ce3b7f33f32a0299493e5884bd
modules/sdk-lib-mpc/src/tss/redpallas-mps/index.ts	353	55817911f7e79b77c1f101729195bc7e5471145746a2b4aa72ed29d68ebfac
modules/sdk-lib-mpc/src/tss/redpallas-mps/rerand.ts	13534	66acb4324fcf192aa4f27f267ae4f1417fe39bfb10b65a00bcf5ab8b6a4ad8
modules/sdk-lib-mpc/src/tss/redpallas-mps/types.ts	6017	c80fc19953e57be021d2e3c0625b9f346481b09ae3c67c2b4c0ff7cdd7ca36
modules/sdk-lib-mpc/src/tss/redpallas-mps/util.ts	8032	6ad2437c5cc6203360e670b414bb0c206f3481bdda95c35b4c20bc846a228e
modules/sdk-lib-mpc/src/types.ts	245	c327c6f7b63601ecce38c87946a9930f3688f4096de2f899877a6b6df5b26f
modules/sdk-lib-mpc/src/util.ts	4696	fa73dabdae9fb6cceb87caad783a84f1aa6538eef5c5818145d51817f44886
package.json	11061	ca643c7512a4c8a5a8448a29f556b95cd7016421b937f6cde0361486fed85f